

Learning-Guided Exact Branch-Price-and-Cut for Multi-Trip Lunar Water-Ice Exploration Routing

Abstract

This paper considers a forward-looking lunar south-pole benchmark in which a rover fleet coordinates in-situ prospecting across spatially dispersed candidate sites, including shadowed cold-trap environments and their surrounding access terrain. The decision assigns sites and predeclared terrain-aware paths to rovers and groups visits into successive depot-to-depot trips. Each trip obeys time-window, energy, load, and cumulative shadow-exposure limits, while return, docking, recharge, and elapsed mission time couple consecutive trips. Waiting is prohibited at candidate task sites and en route, so arrival at a candidate task coincides with the start of its prescribed service. A rover may instead wait at the support depot and adjust the departure time of each trip to reach every selected task within its time window. The prohibition concerns idle loitering; prescribed detection, sampling, and drilling service times remain part of task execution. The problem is formulated over a fixed logical-path solution space, and a pricing-led, branching-assisted learning-guided exact Branch-Price-and-Cut (BPC) framework is specified for its solution. Multi-trip route columns encode complete one-rover schedules. The normalized objective combines operating cost, risk, and **0.4** times science-weighted completion time, so earlier completion is valued more strongly for higher-weight prospecting tasks. Learning ranks pricing work and branch candidates constructed by the exact branching rule. It does not control cuts, bounds, pruning, termination, or proof records. Exact pricing completion, deterministic valid-cut logic, exact branch construction with fail-closed incomplete handling, and the branch tree remain responsible for all formal conclusions. The no-task-wait formulation and its conditional exactness proof are specified in this paper and realized by the frozen exact implementation. In a fresh-process benchmark with one run per instance, all 80 instances with 5–30 tasks reached exact closure and passed the recorded correctness checks under the specified no-task-wait formulation. The learning-guided and seasonal comparisons remain pending. The learning interface is designed to alter only search order, leaving the feasible set, normalized objective, and proof requirements unchanged.

Keywords: lunar exploration routing; multi-trip fleet routing; Branch-Price-and-Cut; resource-constrained shortest path; learning-guided optimization; exact algorithms

1. Introduction

Water is an important resource for sustaining long-term activities on the lunar surface. Evidence from orbital remote sensing, near-infrared observations, and returned-sample studies indicates the presence of water- or water-ice-related signatures on the lunar surface and in polar cold traps [C054; C061; C062]. These observations narrow the areas that may warrant prospecting, but they cannot determine the actual abundance, physical occurrence, or operational accessibility of water at a particular candidate site. Analyses of Chang'E-5 samples further show that lunar surface water can occur in different host materials and that its formation and retention depend on mineral properties, exposure history, and solar-wind interactions [C061; C062]; sites with similar remote-sensing signatures may therefore have different resource states and scientific value. Consequently, the

central task in early lunar water-ice prospecting is not to identify resources from scratch, but to organize rovers to conduct detection, sampling, or drilling within delineated candidate areas and convert remote-sensing anomalies that may indicate water ice into in-situ evidence of resource abundance, physical occurrence, and surface accessibility. Because candidate sites can be numerous and widely dispersed, rapidly planning and executing large-area in-situ prospecting within a finite mission period, while reducing the time required to obtain in-situ evidence from high-value targets, becomes a key issue before lunar in-situ resource utilization can advance toward engineering application.

On the lunar surface, large-area rapid prospecting is shaped jointly by distinctive illumination conditions and complex terrain. Low solar elevations and crater relief at the lunar south pole create permanently shadowed regions (PSRs), illuminated crater rims, and transition areas with markedly different thermal and traversal conditions [C042; C054]. Although PSRs favor the long-term preservation of water ice, entering and leaving them can increase shadow-exposure time—the cumulative time a rover spends without direct sunlight during travel and service—and traversal risk across complex terrain. These effects raise thermal-control demand and energy consumption and make it more difficult to preserve a safe-return margin [C042; C055]. Crater slopes, surface roughness, and shadow exposure jointly determine traversal conditions between prospecting targets. Consequently, several terrain-feasible paths may connect the same pair of sites; although a shorter path reduces spatial distance, it may entail longer shadow exposure and greater traversal risk. Because detection, sampling, and drilling in PSRs can rely on onboard energy, thermal-control systems, and active instruments, direct sunlight is not imposed as a prerequisite for task execution. Instead, predefined task time windows represent constraints on instrument operation, communication scheduling, and mission planning. The rover is not allowed to wait at a candidate site or along a path after leaving the depot. If a selected task would otherwise be reached before its window opens, the corresponding trip is delayed at the support depot; this delay changes the trip's departure time and elapsed mission time but does not create a task-site loitering state. The shift is feasible only if one common departure time satisfies every task window in that trip; otherwise, the selected task sequence and path-option combination is infeasible. The prohibition concerns idle waiting, not the prescribed detection, sampling, or drilling service duration; arrival at a candidate task therefore coincides with service start. The depot is modeled as a base-supported holding environment with power and thermal-control service. Excluding depot waiting from trip energy, path risk, and shadow exposure is therefore a forward-looking benchmark assumption, not a validated hardware capability.

When candidate sites are represented as task nodes, the planning task can be formulated as a multi-path, multi-trip capacitated vehicle-routing problem with time windows and heterogeneous prospecting services. Detection, sampling, and drilling have different service durations, load requirements, and energy demands. Each task also carries a science weight, and delayed completion of a higher-weight task incurs a stronger penalty. In the setting considered here, several alternative paths with different resource attributes may connect any pair of candidate targets. These attributes can differ among path alternatives and independently generated mission-epoch instances, but remain fixed during one optimization run. A feasible plan assigns every task to one rover, partitions each rover's work into a sequence of depot-to-depot trips, selects a path option for every leg, schedules depot waiting and each trip departure, and satisfies every task window without waiting at a candidate site, together with rover load and energy limits, cumulative shadow-exposure limits, return and recharge requirements, and the mission horizon. The experimental setting is explicitly oriented toward future lunar water-ice prospecting. The benchmark instances cover a common **50 km × 50 km** lunar south-polar region constructed from open-source LOLA-derived elevation, slope, roughness, PSR, and average solar-visibility rasters, with the maximum modeled speed set to **30 km h⁻¹**.

Both representing and solving this problem exactly are challenging. Rovers typically need to return to the depot for recharging after completing a subset of tasks and to perform multiple successive mission stages within the planning horizon. At the representation level, compressing the alternative paths between two candidate sites into a single connection discards differences in travel time, energy consumption, risk, and shadow exposure, potentially changing route feasibility and task priority. At the computational level, a route column must encode a complete

multi-trip schedule for one rover rather than a single depot-to-depot trip, and the search space grows exponentially with the number of tasks. The pricing problem must therefore repeatedly search a vast space of task sequences and path choices while maintaining time, energy, load, shadow-exposure, return, and recharge feasibility across successive trips. Related exact-routing studies use route columns to represent complex energy, charging, and onboard production processes [C021; C028; C029; C030]. Learned branching, pricing, and column-generation strategies indicate that data-driven scores can prioritize computationally expensive search decisions [C001; C002; C009], while selective pricing provides a non-learning precedent for ordering pricing effort [C059]. These priorities may move promising work forward in the search order, but they cannot prove that no negative-reduced-cost column remains or that a branch-and-bound node can be closed.

To address these challenges, this paper develops a learning-guided exact Branch-Price-and-Cut (BPC) framework. The learning layer orders pricing work and candidates generated by the exact branching rule. Path feasibility, reduced-cost evaluation, deterministic valid inequalities, node bounds, pruning, and branch-tree closure remain the responsibility of exact procedures; a mandatory exact fallback processes all work that the learning layer postpones or cannot evaluate. In the experimental design, environmental variation will be represented by independently fixed mission-epoch instances. After implementation verification, the same revised solver will be applied separately to each instance to compare seasonal operating phases without adding departure-time-dependent path attributes to the optimization state. Importantly, exactness applies only to the path options declared for each fixed instance within its fixed logical-path solution space. It neither implies optimality over all continuous lunar-surface trajectories nor establishes a single route that remains robust across every environmental phase. The contributions are threefold.

1. Lunar water-ice prospecting is formulated as a multi-path, multi-trip fleet-routing problem that incorporates lunar terrain, cumulative shadow exposure, recharging, heterogeneous services, depot-only waiting, adjustable trip departures, and science-weighted completion time. (2) A learning-guided exact BPC algorithm is developed, together with a conditional proof covering pricing, root-only valid inequalities, branching, pruning, bounds, and branch-tree closure. Learning guides pricing order primarily and branch-candidate order secondarily, without controlling cuts or replacing any exact conclusion. (3) A reproducible lunar benchmark and a correctness-first evaluation package are provided. The frozen no-task-wait baseline establishes exact closure for 80 instances with 5–30 tasks. Paired learning-guidance and four-phase seasonal results are specified by the evaluation protocol but remain pending. The remainder of this paper is organized as follows. Section 2 reviews the related literature. Sections 3 and 4 present the formulation and exact algorithm, respectively. Sections 5 and 6 define the experiments and report the available evidence. Sections 7 and 8 discuss the implications, limitations, and conclusions.

2. Related Work

2.1 Lunar mission planning

Lunar mobility studies address a hierarchy of decisions. Local planners seek a traversable trajectory over topography, while mission-level planners order scientific waypoints and account for illumination, communication, failures, and safe reachability [C041; C042]. Illumination-aware studies construct precomputed connections between south-pole sites [C044], and mission operations concepts combine route choice with terrain, temperature, power, and communication considerations [C055]. Water-ice targeting adds a further spatial layer because thermal stability and illumination help identify candidate prospecting areas [C054]. Together, these studies explain why a south-pole route cannot be reduced to Euclidean distance.

The planning levels remain complementary. Local trajectory generation determines how a rover may move between two locations. Safe mission-level planning determines whether that movement is compatible with environmental and operational conditions. The present problem begins with a finite set of such movement alternatives and then decides which rover serves each prospecting task, how visits are grouped into depot-to-depot trips, how successive trips are coupled through recharge, and which path alternative is used on every leg. This decision scope is narrower than full autonomous mission planning but broader than a single-rover point-to-point traverse.

Map provenance is part of that boundary. The benchmark records topography, slope, roughness, illumination, and permanently shadowed-region layers, then derives finite task and path-option inputs from them. These layers support contrasts among cold-trap interiors, rims, transition areas, and more favorable access terrain; they do not establish ground-truth water-ice abundance. Similarly, BPC exactness can validate the optimum of the resulting discrete model but cannot validate the physical calibration of its risk, energy, or shadow proxies. Continuous path generation, time-dependent environmental prediction, localization, and map updating therefore remain complementary inputs rather than implicit outputs of the proposed optimizer.

2.2 Exact routing algorithms

Route columns are useful when a vehicle's internal operations cannot be represented faithfully by a short arc-flow model. Electric-vehicle routing columns can encode technology choice and recharge decisions [C021], and recent BPC algorithms handle heterogeneous or nonlinear recharge processes through tailored labeling [C029]. Mobile-production routing provides another analogue: route and onboard production schedules are coupled inside one column and solved through branch-price-and-cut [C030]. The transferable principle is to keep exact task coverage and fleet allocation in the master while moving resource-feasible scheduling into the column. In the present setting, that column packages several lunar depot-to-depot trips, their selected path alternatives, cumulative shadow exposure, and recharge transitions.

Pricing, valid inequalities, and branching are established components of exact route-based solvers. The park-and-loop routing literature combines tailored pricing, subset-row inequalities, and branching in an exact BPC [C020]. Time-dependent electric routing uses branch-cut-and-price with resource-aware labeling [C028], while exact pickup-and-delivery research shows how valid inequalities and branching support computational closure even when the main algorithm is branch-and-cut rather than column generation [C023]. Foundational exact-VRP work places these components in the broader development of column generation and branch-cut-and-price [C060]. The contribution claimed here is therefore not BPC, labeling, or subset-row inequalities in isolation. It lies in the route definition and exact integration required when every pricing extension selects a lunar path option and updates time, energy, load, shadow exposure, risk, science-weighted completion, and multi-trip recharge state under one reduced-cost definition.

Pricing control must also be distinguished from pricing exhaustion. Selective pricing reduces computational effort by deciding when a relaxed pricing problem is sufficient and when a stronger problem should be solved [C059]. That principle motivates both the deterministic comparator and the learned ordering layer used here. Yet an ordered or relaxed search can stop early only as a heuristic stage. The final no-negative-column conclusion still depends on the exact pricing procedure and on its coverage of the complete admissible multi-trip route space under the active lunar-resource, cut, and branch context.

2.3 Learning-guided optimization

Learning-to-branch methods use graph, global, and historical solver features to imitate or improve expensive branching rules [C003]. More directly, two-stage learning has been incorporated into an exact vehicle-routing BPC [C001]. That precedent rules out a broad claim that learned branching in exact routing is new. The paper-specific question is whether a lunar multi-trip route state, an exact-valid candidate interface, and deterministic fallback can be combined without transferring branch validity or completeness to the learned model.

Learned pricing follows a parallel line. Reinforcement learning has been used to address column-generation pricing for routing [C002], and reinforcement-learning hyper-heuristics have controlled pricing heuristics in routing and scheduling [C009]. The broader GNN literature distinguishes neural combinatorial solvers from neural components that enhance an exact solver [C008]. The architecture adopted here belongs to the second category. Predictions rank task/path extensions and related pricing work, whereas the exact solver decides route admissibility and exhausts the required pricing space before any proof-bearing bound is accepted.

Learning to control cuts is intentionally outside the study. Deterministic subset-row inequalities may remain active because their validity, coefficients, dual contributions, and lifecycle can be audited against exact pricing. No learned component generates, selects, activates, retains, or deletes a cut. The resulting gap lies at the intersection of these three streams. Lunar planners provide terrain-aware movement alternatives, exact routing methods provide route-level bounds and optimality proofs, and learned solver control provides workload priorities, but their integration for repeated, exposure-constrained rover trips has not been established by the cited work. The proposed framework addresses this intersection while retaining exhaustive pricing and deterministic proof operations. Exact-equivalence, workload, overhead, fallback, and held-out evidence are still required to evaluate the learning component; those evidence groups remain pending in the present working draft.

3. Problem Definition and Mathematical Formulation

The model represents a regional prospecting campaign over one frozen **50 km × 50 km** lunar south-pole base map. Candidate tasks are sampled from areas with high values of the frozen resource-index proxy, permanently shadowed-region boundaries, and surrounding exploration areas, with detection, sampling, and drilling as distinct service modes. The spatial extent, mobility parameters, task attributes, and time horizons define a forward-looking benchmark scenario; they are not measurements of current rover performance or evidence of in-situ water-ice abundance.

Three decision objects are separated. A path option is one precomputed way to traverse a directed task-network edge. A trip leaves the support depot, serves one or more tasks, and returns. A multi-trip route is the time-compatible sequence of one or more such trips assigned to one rover, including the docking and recharge transitions between trips; it is the column selected by the master problem. Keeping these objects distinct prevents terrain-level movement, resource-feasible prospecting trips, and fleet-level schedules from being conflated. Section 3.1 defines the fixed network and path options, Section 3.2 constructs feasible trips and multi-trip routes, and Section 3.3 gives the normalized objective and route-based master problem.

3.1 Lunar task network and path options

Let $\mathcal{T}$ be the set of prospecting tasks, $\mathcal{K}$ the rover set, and $\mathbf{0}$ the depot. The mission-level network is a directed task graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{\mathbf{0}\} \cup \mathcal{T}$. Each task $i \in \mathcal{T}$ has a location, operation mode, science weight w_i , load q_i , service duration σ_i , service energy g_i , service cost c_i^{srv} , time window $[r_i, D_i]$, local shadow score η_i , and frozen service-risk contribution ρ_i^{srv} . The operation mode identifies whether the declared in-situ task is detection, sampling, or drilling. These quantities are frozen inputs. Here r_i is the earliest admissible service-start time and D_i is the latest admissible service-completion time, so a task must start no later than $D_i - \sigma_i$. The service-risk contribution is computed before optimization from the recorded task thermal-risk metadata and prescribed service duration; its internal benchmark conversion is not a free model coefficient. The optimization model neither estimates water-ice abundance nor updates the environmental map during a solve. The admissible input domain is

$$w_i, q_i, \sigma_i, g_i, c_i^{\text{srv}}, \rho_i^{\text{srv}}, \eta_i \geq 0 \quad (i \in \mathcal{T}),$$

and the three normalization references defined in (9) are strictly positive. These sign conditions are part of the exact-model contract rather than empirical performance claims.

Let $\mathcal{Q}$ contain the selected mission epochs used in environmental scenario construction, and let $\mathcal{I}^\zeta$ denote the independently generated instance associated with epoch $\zeta \in \mathcal{Q}$. Let b_ζ be the epoch anchor and $\mathcal{W}_\zeta = [b_\zeta, b_\zeta + H^{\text{mis}}]$ its mission window. The anchor interval and H^{mis} have different roles: the first samples environmental phases, whereas the second limits task execution inside one instance. The planned M006 protocol starts from the southern vernal equinox and uses **12** anchors uniformly spaced over one draconic year, about **28.9** Earth days apart, with three anchors in each south-polar spring, summer, autumn, and winter phase and an environmental sampling interval $\Delta^{\text{env}} = 1$ h. This design draws on the hourly resolution and 12-lunation coverage reported by Kloos et al. [C063]. The hourly states in $\mathcal{W}_\zeta$ are converted by one declared preprocessing rule into the fixed illumination and related environmental layers supplied to path generation. This rule and its data hashes must be frozen in M006; they are not decisions of the routing model.

Terrain, task locations, rover parameters, the scale-dependent mission horizon, and normalization references are held common across epochs in controlled comparisons, whereas the generated path-option set and its travel-time, energy, risk, and shadow attributes may differ. Epoch ζ is an instance-generation index, not an optimization decision or an SPPRC state. All subsequent formulation and proof statements condition on one fixed $\mathcal{I}^\zeta$, and the superscript ζ is omitted from path quantities to keep the route model readable.

For each $(u, v) \in \mathcal{E}$, the fixed epoch-conditioned instance declares a finite $\mathcal{A}_{uv}$ containing three path alternatives, respectively selected to prioritize minimum travel time, minimum energy consumption, and minimum traversal risk. Option $\omega \in \mathcal{A}_{uv}$ carries travel time τ_{uv}^ω , energy proxy e_{uv}^ω , integrated risk ρ_{uv}^ω , distance d_{uv}^ω , shadow exposure h_{uv}^ω , and a precomputed geometry. The optimizer selects among these alternatives at runtime; it does not generate a continuous surface trajectory. The distinction is operationally important: the lowest-time alternative need not be the lowest-shadow or lowest-risk alternative, so mission-level feasibility cannot be inferred from geometric distance alone. Every declared option satisfies $\tau_{uv}^\omega, e_{uv}^\omega, \rho_{uv}^\omega, d_{uv}^\omega, h_{uv}^\omega \geq 0$. These input-domain conditions are checked before proof-bearing pricing.

The stored quantities are constructed from lunar-surface layers before optimization. For a directed path option $\ell = (u, v, \omega)$ sampled at grid locations $\mathbf{x}_{\ell 1}, \dots, \mathbf{x}_{\ell N_\ell}$ with cell width Δ_{km} , the path distance and the mean of a sampled surface layer $\phi(\mathbf{x})$ are

$$d_\ell = \Delta_{\text{km}} \sum_{k=1}^{N_\ell-1} \|\mathbf{x}_{\ell, k+1} - \mathbf{x}_{\ell k}\|_2, \quad \bar{\phi}_\ell = \frac{1}{N_\ell} \sum_{k=1}^{N_\ell} \phi(\mathbf{x}_{\ell k}). \quad (1)$$

For the selected epoch, preprocessing summarizes slope, roughness, mission-window illumination, permanently shadowed-region structure, crater proximity, steep-slope exposure, and directional elevation change along each declared geometry. Permanent-shadow membership and topography are spatial inputs; the epoch-conditioned window summary describes transient lighting outside those regions without making lighting an optimization state. Preprocessing then maps these summaries to the stored travel time τ_ℓ , energy proxy e_ℓ , integrated risk ρ_ℓ , and shadow exposure h_ℓ . These four quantities are inputs to the optimization model rather than quantities fitted or estimated by BPC. The exact solver therefore treats $(d_\ell, \tau_\ell, e_\ell, \rho_\ell, h_\ell)$ as immutable inputs. Spatial variation is retained through these option-specific values, whereas their evolution during a trip is not predicted online. This frozen-input assumption is needed both for reproducibility and for the exactness proof in Section 4.7. It proves exactness for the resulting window-aggregated instance, not physical equivalence to a continuously evolving illumination field.

The numerical mixing coefficients used by the benchmark generator are not displayed as model equations because the available materials do not establish their physical calibration or sensitivity. Reproducibility instead relies on the frozen generator source, configuration, and stored path-option records.

The no-task-wait rule restricts path-option preprocessing. A faster path cannot automatically dominate a slower path, because the additional travel time of the slower option may be needed to reach a task after its release time without waiting away from the depot. Delaying the whole trip at the depot does not make such a slower internal leg redundant: a common departure shift also delays every earlier task in that trip and can violate one of their due times. Pricing may therefore remove option $\omega_b \in \mathcal{A}_{uv}$ only if a retained option $\omega_a \in \mathcal{A}_{uv}$ has exactly the same travel time,

$$\tau_{uv}^{\omega_a} = \tau_{uv}^{\omega_b}, \quad (e_{uv}^{\omega_a}, \rho_{uv}^{\omega_a}, d_{uv}^{\omega_a}, h_{uv}^{\omega_a}) \leq (e_{uv}^{\omega_b}, \rho_{uv}^{\omega_b}, d_{uv}^{\omega_b}, h_{uv}^{\omega_b}),$$

componentwise, with at least one strict inequality. The substitution then preserves every task arrival and completion time while weakly reducing all other path resources and objective components. Options with unequal travel times remain in the proof-bearing path set even when one appears componentwise preferable under the remaining attributes. In the mathematical model, equality of travel time is exact. The executable uses recorded numerical tolerances for this comparison. The proof-bearing native completion uses 10^{-12} , whereas the Python seed and reference constructor uses 10^{-9} and cannot establish the absence of a negative-reduced-cost route. The official closure claim is therefore bound to the native comparison and its recorded tolerance; a looser approximate comparison would not be proof preserving.

Define $\mathcal{R}(\mathcal{I})$ as the set of feasible one-rover multi-trip routes induced by the frozen task graph, path options, tasks, resources, time windows, and objective parameters of instance $\mathcal{I}$. Define $\Omega(\mathcal{I})$ as the feasible fleet schedules formed by selecting at most $|\mathcal{K}|$ members of $\mathcal{R}(\mathcal{I})$ so that every task is covered exactly once. Every exact or optimal statement refers to this fixed logical-path solution space. Environmental fidelity remains a separate question: an exact solution of the discrete model does not prove that its terrain, illumination, risk, or resource layers fully represent future surface conditions. In particular, applying the solver separately to several $\mathcal{I}^\zeta$ instances proves the result only for each corresponding window-aggregated instance; it does not prove that one route is robust or dynamically optimal across all $\zeta \in \mathcal{Q}$.

Figure 1 placeholder (FIG01/FIG06, evidence available): lunar south-pole planning layers and one representative fixed task graph with three declared path alternatives per directed edge. The final caption must distinguish measured inputs, derived proxies, and visualization-only layers.

3.2 Multi-trip route feasibility

In the benchmark, every trip leaves and returns to the support depot. Load capacity, usable energy, and the shadow-exposure allowance are enforced for each trip, whereas a later trip may begin only after the preceding return, docking, and recharge transition is complete. Waiting is prohibited at candidate task sites and en route. It is allowed only at the support depot, where the departure of any trip, including the first, may be delayed. Depot waiting contributes to elapsed mission time and the mission horizon. The support depot is modeled as a controlled holding environment with base-provided power and thermal control, so the rover does not accumulate trip load, travel energy, path risk, or off-depot shadow exposure during that delay. Base-supplied standby energy and thermal-control demand are outside the present system boundary. This supported-depot standby assumption is a forward-looking benchmark assumption rather than a claim about validated lunar infrastructure. The prohibition applies to idle loitering, not to the prescribed service duration of a detection, sampling, or drilling task. Accordingly, feasibility is enforced first within each depot-to-depot trip and then across the ordered trips assigned to the same rover. A trip s starts at the depot, visits an ordered sequence of distinct tasks, and returns to the depot:

$$s = (0, i_1, \dots, i_m, 0; \omega_0, \dots, \omega_m; t_s^0), \quad m \leq M, \quad (2)$$

where M is the maximum number of tasks per trip, t_s^0 is its departure time, and each ω_j , $j = 0, \dots, m$, selects the path option for the corresponding directed leg.

For a fixed sequence and fixed path options, let $\delta_{j,s}$ be the elapsed travel-and-service time from the trip departure to the start of task i_j . Let a_s be the earliest time at which the rover is available at the depot: $a_1 = 0$ and $a_s = t_{s-1}^{\text{end}}$ for $s \geq 2$. The fixed duration from departure through return, docking, and recharge is denoted by χ_s . These quantities and the feasible interval for the trip departure are given below. Each task window is shifted backward by its fixed departure-to-service offset; their intersection gives all depot departures that require no later waiting.

$$\begin{aligned} \delta_{1,s} &= \tau_{0,i_1}^{\omega_0}, \\ \delta_{j,s} &= \delta_{j-1,s} + \sigma_{i_{j-1}} + \tau_{i_{j-1},i_j}^{\omega_{j-1}}, & (j = 2, \dots, m), \\ \chi_s &= \delta_{m,s} + \sigma_{i_m} + \tau_{i_m,0}^{\omega_m} + d^{\text{dock}} + \frac{E_s}{P^{\text{rch}}}, \\ \underline{t}_s^0 &= \max \left\{ a_s, \max_{j=1, \dots, m} (r_{i_j} - \delta_{j,s}) \right\}, \\ \bar{t}_s^0 &= \min \left\{ \min_{j=1, \dots, m} (D_{i_j} - \sigma_{i_j} - \delta_{j,s}), H^{\text{mis}} - \chi_s \right\}, \\ \underline{t}_s^0 &\leq \bar{t}_s^0, \quad t_s^0 = \underline{t}_s^0, \quad \Delta_s^{\text{dep}} = t_s^0 - a_s, \\ t_{i_j,s}^{\text{arr}} &= t_{i_j,s}^{\text{start}} = t_s^0 + \delta_{j,s}, \quad t_{i_j,s}^{\text{cmp}} = t_{i_j,s}^{\text{start}} + \sigma_{i_j}. \end{aligned} \quad (3)$$

The interval is nonempty exactly when the fixed trip can meet its temporal conditions—every task window, preceding-trip availability, and the mission horizon—without waiting at a task or en route. The canonical departure $t_s^0 = \underline{t}_s^0$ places any necessary delay at the depot and gives the earliest feasible completion times for that fixed sequence and path choice. A later departure inside the interval is feasible but cannot improve the objective because every science weight is nonnegative.

For exposition and independent verification, the following compact formulation defines the internal feasibility of a multi-trip route column; the BPC master does not retain these trip-level arc variables. Let $\mathcal{S} = \{1, \dots, \bar{S}\}$ be the ordered set of potential trip slots in one multi-trip route. Because every active trip contains at least one task and a task occurs in at most one trip of the route, setting $\bar{S} = |\mathcal{T}|$ is a valid nonrestrictive upper bound. Let $\mathcal{L} = \{(u, v, \omega) : (u, v) \in \mathcal{E}, \omega \in \mathcal{A}_{uv}\}$ be the set of directed path-option arcs. Let $x_{\ell s} = 1$ if option arc ℓ is used in slot s , $y_{is} = 1$ if task i is visited in that slot, and $z_s = 1$ if the slot is active. The sets $\delta^-(v)$ and $\delta^+(v)$ contain the incoming and outgoing path-option arcs of vertex v . Depot degrees, task flow balance, trip activation, route-level task uniqueness, and consecutive use of trip slots are imposed by

$$\begin{aligned} \sum_{\ell \in \delta^+(0)} x_{\ell s} &= z_s = \sum_{\ell \in \delta^-(0)} x_{\ell s} & (s \in \mathcal{S}), \\ \sum_{\ell \in \delta^-(i)} x_{\ell s} &= y_{is} = \sum_{\ell \in \delta^+(i)} x_{\ell s} & (i \in \mathcal{T}, s \in \mathcal{S}), \\ z_s &\leq \sum_{i \in \mathcal{T}} y_{is} \leq M z_s & (s \in \mathcal{S}), \\ \sum_{s \in \mathcal{S}} y_{is} &\leq 1 & (i \in \mathcal{T}), \\ 1 &\leq \sum_{s \in \mathcal{S}} z_s, \quad z_{s+1} \leq z_s & (s = 1, \dots, \bar{S} - 1). \end{aligned} \tag{4a}$$

The corresponding variable domains are

$$x_{\ell s} \in \{0, 1\}, \quad y_{is} \in \{0, 1\}, \quad z_s \in \{0, 1\}, \quad t_{is}^{\text{start}}, t_{is}^{\text{cmp}}, t_s^0, t_s^{\text{return}}, t_s^{\text{rch}}, t_s^{\text{end}}, \Delta_s^{\text{dep}} \geq 0 \quad (\ell \in \mathcal{L}, i \in \mathcal{T}, s \in \mathcal{S}). \tag{4b}$$

Flow balance alone permits a disconnected task cycle. Trip connectivity and elementarity can be expressed by the classical subtour-elimination family

$$\sum_{\substack{\ell=(u,v,\omega): \\ u,v \in U}} x_{\ell s} \leq |U| - 1, \quad \emptyset \neq U \subseteq \mathcal{T}, \quad s \in \mathcal{S}. \tag{5}$$

Here U is a task subset rather than a trip index. Equations (4a)–(5) are not additional rows of the route master. They define the internal topology of every feasible column in $\mathcal{P}(n)$. The compact reference MILP may replace (5) by equivalent single-commodity-flow or Miller–Tucker–Zemlin constraints. The native SPPRC enforces elementarity constructively by carrying the visited-task set and rejecting a repeated-task extension. Thus, removing trip-level arc variables from the master does not remove flow conservation or subtour prevention from the algorithm.

Time-window feasibility and no-wait temporal propagation are the next defining constraint family. Completion is linked linearly to service start, while every selected path-option arc activates a temporal equality. The equalities prevent unmodeled idle time from appearing at a candidate task or along a leg:

$$\begin{aligned} r_i y_{is} &\leq t_{is}^{\text{start}} \leq (D_i - \sigma_i) y_{is}, \quad t_{is}^{\text{cmp}} = t_{is}^{\text{start}} + \sigma_i y_{is} & (i \in \mathcal{T}, s \in \mathcal{S}), \\ x_{(0,j,\omega),s} = 1 &\Rightarrow t_{js}^{\text{start}} = t_s^0 + \tau_{0j}^\omega & (j \in \mathcal{T}, \omega \in \mathcal{A}_{0j}, s \in \mathcal{S}), \\ x_{(i,j,\omega),s} = 1 &\Rightarrow t_{js}^{\text{start}} = t_{is}^{\text{cmp}} + \tau_{ij}^\omega & (i, j \in \mathcal{T}, \omega \in \mathcal{A}_{ij}, s \in \mathcal{S}), \\ x_{(i,0,\omega),s} = 1 &\Rightarrow t_s^{\text{return}} = t_{is}^{\text{cmp}} + \tau_{i0}^\omega & (i \in \mathcal{T}, \omega \in \mathcal{A}_{i0}, s \in \mathcal{S}), \\ z_s = 1 &\Rightarrow 0 \leq t_s^0 \leq t_s^{\text{return}} \leq t_s^{\text{end}} \leq H^{\text{mis}} & (s \in \mathcal{S}), \\ z_1 = 1 &\Rightarrow t_1^0 = \Delta_1^{\text{dep}}, \\ z_{s+1} = 1 &\Rightarrow t_{s+1}^0 = t_s^{\text{end}} + \Delta_{s+1}^{\text{dep}} & (s = 1, \dots, \bar{S} - 1). \end{aligned} \tag{6a}$$

The implications in (6a) are standard MILP indicator constraints. Each selected-arc equality can be represented by a pair of valid big- M rows derived from the mission horizon, task time windows, and path travel times; those bounds are not calibrated lunar-performance coefficients. The variable Δ_s^{dep} measures only waiting at the support depot. No

task-site or en-route waiting variable is present. Service-start and completion times differ by the prescribed service duration, which is task execution rather than waiting.

The additive trip resources and objective components are reconstructed from the same arc and visit variables:

$$\begin{aligned}
Q_s &= \sum_{i \in \mathcal{T}} q_i y_{is}, \\
E_s &= \sum_{\ell \in \mathcal{L}} e_\ell x_{\ell s} + \sum_{i \in \mathcal{T}} g_i y_{is}, \\
H_s &= \sum_{\ell \in \mathcal{L}} h_\ell x_{\ell s} + \sum_{i \in \mathcal{T}} \eta_i \sigma_i y_{is}, \\
R_s &= \sum_{\ell \in \mathcal{L}} \rho_\ell x_{\ell s} + \sum_{i \in \mathcal{T}} \rho_i^{\text{srv}} y_{is}, \\
C_s &= \sum_{i \in \mathcal{T}} c_i^{\text{srv}} y_{is} + \sum_{\ell \in \mathcal{L}} d_\ell x_{\ell s} + E_s, \\
T_s^w &= \sum_{i \in \mathcal{T}} w_i t_{is}^{\text{cmp}}.
\end{aligned} \tag{6b}$$

Thus, H_s is the cumulative off-depot shadow exposure accumulated across movement and service within a trip, whereas integrated traversal and service risk enters the objective. The task term ρ_i^{srv} is an immutable preprocessing input rather than a coefficient calibrated inside the optimization model. Depot waiting is excluded from Q_s , E_s , H_s , R_s , and C_s but is included in all absolute task-completion times and in the mission horizon through t_s^0 . After the return leg, the resource limits, recharge duration, trip end, and mission horizon are

$$\begin{aligned}
Q_s &\leq Q z_s, & E_s &\leq B z_s, & H_s &\leq H^{\max} z_s, \\
t_s^{\text{rch}} &= d^{\text{dock}} z_s + \frac{E_s}{P^{\text{rch}}}, & t_s^{\text{end}} &= t_s^{\text{return}} + t_s^{\text{rch}}, \\
t_s^{\text{end}} &\leq H^{\text{mis}} z_s & (s \in \mathcal{S}).
\end{aligned} \tag{7}$$

Here d^{dock} is a docking overhead, P^{rch} is the recharge-power proxy, Q is rover capacity, B the usable-energy limit, $H^{\max}$ the trip shadow-exposure limit, and H^{mis} the mission horizon. Their input domain satisfies $d^{\text{dock}} \geq 0$ and $P^{\text{rch}}, Q, B, H^{\max}, H^{\text{mis}} > 0$.

The shadow limit and the risk term have deliberately different mathematical roles. A trip is infeasible when its cumulative shadow exposure exceeds $H^{\max}$ even if its energy, capacity, and time-window constraints would otherwise hold. By contrast, integrated risk discriminates among feasible routes through the objective and is not silently converted into a feasibility threshold. Although the horizon-ordering inequalities alone are weak, the selected-arc and recharge equalities uniquely determine return and trip-end times once t_s^0 is fixed. Equation (3) then chooses the earliest feasible depot departure. Earlier depot availability cannot harm a later trip because the rover may remain at the depot, whereas waiting at a task or en route is never admitted.

Equations (4a)–(7) give the core trip-level MILP families that define a feasible multi-trip route column. Pairwise incompatibility cuts, cover inequalities, slot-position bounds, and tighter big- M values used by the compact verifier are implementation strengthenings rather than omitted defining constraints. Their computational effects require separate evaluation.

A multi-trip route $p = (s_1, \dots, s_{m_p})$ is one rover's ordered schedule. Every constituent trip is feasible, the task sets are disjoint, and $t_{s_{j+1}}^0 = t_{s_j}^{\text{end}} + \Delta_{s_{j+1}}^{\text{dep}}$ for $j = 1, \dots, m_p - 1$. Let $\mathcal{T}_p$ be the tasks served by p , with $a_{ip} = 1$ when $i \in \mathcal{T}_p$ and 0 otherwise. One selected multi-trip route consumes one rover. It is neither a single trip nor one local path. The route-level quantities are additive across its task-disjoint trips:

$$C_p = \sum_{s \in p} C_s, \quad R_p = \sum_{s \in p} R_s, \quad T_p^w = \sum_{s \in p} T_s^w = \sum_{i \in \mathcal{T}_p} w_i t_{ip}^{\text{cmp}}. \tag{8}$$

3.3 Route-based master problem

The objective has one form throughout the manuscript. Let C_p denote route operating cost, comprising service cost, path distance, and energy proxy; let R_p denote route and service risk; and let T_p^w denote science-weighted completion time as defined in (8). The first two terms distinguish resource use and exposure among feasible fleet schedules. The third makes the order of prospecting consequential: completing a task earlier reduces the objective, and the reduction is larger for a task with greater science weight. It does not replace exact task coverage, and it is not a makespan term. The normalizers are built from componentwise best feasible single-task routes. If $(\hat{C}_i, \hat{R}_i, \hat{T}_i^w)$ denotes those single-task reference values and $\varepsilon_0 = 10^{-9}$ is the positive floor, then

$$\begin{aligned} C^{\text{ref}} &= \max \left\{ \varepsilon_0, \sum_{i \in \mathcal{T}} \hat{C}_i \right\}, \\ R^{\text{ref}} &= \max \left\{ \varepsilon_0, \sum_{i \in \mathcal{T}} \hat{R}_i \right\}, \\ T^{\text{w}, \text{ref}} &= \max \left\{ \varepsilon_0, \sum_{i \in \mathcal{T}} \hat{T}_i^w \right\}. \end{aligned} \quad (9)$$

An infeasible single-task reference invokes the implemented positive fallback and increments an explicit provenance counter; it is not silently omitted. For the fixed positive quantities in (9), the route cost is

$$c_p = \frac{C_p}{C^{\text{ref}}} + \frac{R_p}{R^{\text{ref}}} + 0.4 \frac{T_p^w}{T^{\text{w}, \text{ref}}}. \quad (10)$$

Column construction, the restricted master, proof-bearing pricing, objective closure, tables, and later translations must use (10). Makespan does not enter this objective. It is reported after selection as

$$M^{\text{rep}} = \max_{p: \lambda_p=1} \max_{i \in \mathcal{T}_p} t_{ip}^{\text{cmp}}. \quad (11)$$

Optimizing makespan would require a different master model with an explicit linking variable and corresponding constraints.

Let $\mathcal{P}(n) \subseteq \mathcal{R}(\mathcal{I})$ be the feasible multi-trip route set under the branch context of node n , and let λ_p select route p . With active deterministic valid inequalities $\mathcal{H}(n)$, the integer master is

$$\begin{aligned} \min_{\lambda} \quad & \sum_{p \in \mathcal{P}(n)} c_p \lambda_p \\ \text{s. t.} \quad & \sum_{p \in \mathcal{P}(n)} a_{ip} \lambda_p = 1, \quad i \in \mathcal{T}, \\ & \sum_{p \in \mathcal{P}(n)} \lambda_p \leq |\mathcal{K}|, \\ & \sum_{p \in \mathcal{P}(n)} a_{hp} \lambda_p \leq b_h, \quad h \in \mathcal{H}(n), \\ & \lambda_p \in \{0, 1\}, \quad p \in \mathcal{P}(n). \end{aligned} \quad (12)$$

The task rows enforce exact coverage, the fleet row limits the selected routes, and a_{hp} is the coefficient of route p in a valid cut h . Learning does not determine any of these rows.

The restricted master problem (RMP) is the LP relaxation over a finite $\mathcal{P}'(n) \subseteq \mathcal{P}(n)$ used during column generation. If π_i , μ , and γ_h are the duals of the task, fleet, and active-cut rows, then

$$\bar{c}_p = c_p - \sum_{i \in \mathcal{T}} \pi_i a_{ip} - \mu - \sum_{h \in \mathcal{H}(n)} \gamma_h a_{hp}. \quad (13)$$

Branch decisions are feasibility conditions defining $\mathcal{P}(n)$; they are not dual terms. Eq. (13) is shared by pricing, column admission, and reduced-cost audits, which is essential when a learned layer can change work order but not column validity.

4. Learning-Guided Exact Branch-Price-and-Cut Framework

4.1 Algorithm overview

The route master sees complete one-rover schedules, not individual lunar movements. Pricing is therefore the operational core of the decomposition: it must assemble prospecting tasks and terrain-aware path options into multi-trip routes that satisfy the full time, load, energy, shadow, docking, and recharge logic of Section 3. The algorithm must find useful routes early enough to make the decomposition practical while still exhausting this admissible route space whenever a node bound or tree conclusion is required.

The proposed framework has two operational lanes but one mathematical model. The guidance lane assigns priorities to pricing work and to branch candidates constructed by the exact branching rule. The exact lane solves the restricted master, checks column addability, performs proof-bearing complete pricing, constructs valid cuts and branch children, maintains bounds, and writes proof records. For the revised timing model, complete pricing becomes proof-bearing only after implementation verification confirms every no-task-wait transition and state invariant. Figure 2 summarizes this asymmetric responsibility. Information moves from exact state records to the guidance layer and returns only as validated priority scores bound to the current solver state; no learned output is connected to the cut lifecycle, node bound, pruning decision, or audited proof records.

Figure 2 placeholder (FIG09/FIG10, method evidence available): two-lane architecture. The learning lane contains pricing-work ordering and valid branch-candidate ranking. The exact lane contains RMP optimization, column validity, proof-bearing exact completion, deterministic cuts, branch construction and fallback, bounds, pruning, and proof records. No arrow is permitted from learning to cut control or proof state.

At a branch-tree node, the solver follows four phases. First, it solves the current RMP and extracts the true dual vector. Second, ordered or fast pricing searches for useful columns. Third, proof-bearing exact completion is invoked whenever proof-producing closure is required. Finally, the solver adds columns, strengthens the root RMP with deterministic valid cuts, accepts an exact node bound, or branches. A resource limit ends this sequence as incomplete. In particular, termination before the pricing frontier has been exhausted cannot support a proof that no negative-reduced-cost column exists. Algorithm 1 expands these phases into the node-control procedure used throughout this section.

Algorithm 1. Exact processing of one BPC node with guidance-only ordering

Require: node n ; branch context $\mathcal{B}(n)$; active deterministic-cut context $\mathcal{H}(n)$; restricted route pool $\mathcal{P}'(n)$; incumbent upper bound U ; exact resource limits; optional typed pricing and branch-ranking hints.

Ensure: an audited node outcome and any exact-admissible columns, cuts, or child contexts.

Line	Procedure
1	Initialize the set of unresolved deferred-pricing obligations $\mathcal{D} \leftarrow \emptyset$.
2	while node n is unresolved do
3	Solve the current RMP; if it is infeasible after a context change, run exact Phase I under the same $\mathcal{B}(n)$ and $\mathcal{H}(n)$.
4	if Phase I neither restores feasibility nor proves node infeasibility then terminate node processing without an exact conclusion.
5	Extract the true RMP duals and audit the reduced costs and context bindings of all active columns.
6	if the RMP is not optimal or any audit fails then terminate node processing without an exact conclusion.
7	Run guidance-ordered pricing with exact completion by Algorithm 2.

Line	Procedure
8	if Algorithm 2 finds one or more exact-addable negative-reduced-cost columns then admit them and continue .
9	if Algorithm 2 terminates before proving pricing closure then terminate node processing without an exact conclusion.
10	At the root only, separate the predefined task-triple valid inequalities described in Section 4.4; learning supplies no cut action.
11	if one or more cuts are admitted then update $\mathcal{H}(n)$ and continue .
12	Record the RMP objective as a valid node lower bound only after exhaustive pricing has proved that no negative-reduced-cost column exists and $\mathcal{D} = \emptyset$.
13	if the lower bound excludes improvement over the incumbent then fathom the node by bound.
14	if the RMP solution is integral then accept it as an incumbent candidate and fathom the node.
15	Run exact-valid branch-candidate ranking and child construction by Algorithm 3.
16	if two valid child contexts are constructed then pass both children to the tree search; otherwise terminate node processing without an exact conclusion.
17	end while

Lines 3–6 establish an optimal and audited RMP state. Lines 7–9 delegate only the order of pricing work while retaining exact completion. Lines 10–12 place deterministic separation before the node bound is declared valid. Lines 13–16 then fathom, accept an incumbent, or branch; no learned output writes any of these outcomes.

Let L_n be the audited node lower bound, U the incumbent objective, z_n^{LB} indicate that all conditions required for a valid node lower bound have been satisfied, z_n^{price} indicate true-dual pricing closure, z_n^{audit} indicate successful context and reduced-cost audits, and $\mathcal{D}_n$ be the set of unresolved deferred-pricing obligations. Bound pruning is permitted only under the conjunction

$$\text{PRUNE}_{\text{bound}}(n) = \begin{cases} 1, & \text{if } z_n^{\text{LB}} = z_n^{\text{price}} = z_n^{\text{audit}} = 1, \quad \mathcal{D}_n = \emptyset, \quad \text{and} \quad L_n \geq U - \varepsilon_{\text{bnd}}, \\ 0, & \text{otherwise.} \end{cases} \quad (14)$$

Thus, the node may be fathomed by bound only when every condition in the first line of (14) holds. The present algorithm applies this test to nonroot nodes. A restricted-master value, diagnostic lower bound, incomplete pricing pass, or nonempty deferred-pricing set makes at least one condition in (14) false and therefore cannot fathom the node.

The same responsibility boundary governs every proof-bearing statement. Learned scores are diagnostic or heuristic signals. Exact pricing may prove the absence of negative columns only after exhaustive coverage under the active context, and the branch tree may prove optimality only when every open node has been processed by a valid bound, infeasibility proof, or integral incumbent condition. The distinction is functional rather than rhetorical: each proof-bearing event has a source record that learned code is not allowed to create or mutate.

4.2 Restricted master problem

The RMP begins from a finite set of feasible multi-trip route columns. Each column already contains its lunar path-option choices, prospecting-task completion times, resource totals, and recharge transitions. After solving the LP, the solver audits the dual vector and reconstructs reduced costs of active columns using (13). Pricing receives the same objective references, task duals, fleet dual, active deterministic-cut duals, branch context, and instance binding. A generated route enters the pool only if its resources and branch conditions are feasible, its deterministic-cut coefficients are consistent, and its true reduced cost satisfies the admission rule.

The negative-column harvest makes that admission rule explicit. Let $\mathcal{C}_n$ be the raw candidate pool, and let $I_p^{\text{br}}, I_p^{\text{cut}}, I_p^{\text{uniq}}, I_p^{\text{add}} \in \{0, 1\}$ indicate branch feasibility, cut-context consistency, signature uniqueness, and master addability, respectively. Let $I_p^{\text{uniq}} = 1$ only for the canonical representative of a route signature within the raw candidate batch. Let $I_p^{\text{add}} = 1$ only when that signature is absent from the current RMP and is either absent from the persistent pool or present there with a matching coefficient/context record that is eligible for reactivation or replacement. Let g_p be an accepted guidance priority, with $g_p = 0$ under deterministic harvesting, and let $\mathcal{S}_n^{\text{act}}$ be the task sets already active in the RMP. If $\kappa_p = (-g_p, \bar{c}_p, \mathcal{T}_p)$ is the lexicographic ordering key, the implemented selection can be written as

$$\begin{aligned} \mathcal{A}_n &= \{p \in \mathcal{C}_n : \bar{c}_p < -\varepsilon_{\text{rc}}, I_p^{\text{br}} I_p^{\text{cut}} I_p^{\text{uniq}} I_p^{\text{add}} = 1\}, \\ \mathcal{N}_n &= \text{FirstPerTaskSet}_{\kappa}(\{p \in \mathcal{A}_n : p \text{ changes active support, } \mathcal{T}_p \notin \mathcal{S}_n^{\text{act}}\}), \\ \mathcal{H}_K(n) &= \text{First}_K(\text{sort}_{\kappa}(\mathcal{N}_n) \parallel \text{sort}_{\kappa}(\mathcal{A}_n \setminus \mathcal{N}_n)). \end{aligned} \quad (15)$$

Here $\parallel$ denotes ordered concatenation. The first block retains at most the first representative of each task set not already active in the RMP; the second contains replacements and remaining addable negatives. Consequently, guidance can change κ_p and hence discovery order, but it cannot make a nonnegative, branch-infeasible, cut-inconsistent, noncanonical batch duplicate, or nonaddable column enter $\mathcal{H}_K(n)$.

Duplicate handling is part of correctness. Repeated signatures within the raw batch are reduced to one canonical candidate, and a signature already active in the RMP is not addable. A signature stored in the persistent pool but absent from the current RMP may be reactivated only when its coefficient and context record matches. Otherwise it reveals an RMP membership inconsistency, a coefficient mismatch, or an outdated column; that state blocks node closure. Learning may decide which candidate is inspected first, but it cannot change the signature, feasibility test, coefficient vector, reduced cost, reactivation eligibility, or replacement rule.

The same binding is retained when cuts are active. Phase-I artificial variables can restore an RMP that is temporarily infeasible under a new exact context, but they do not consume fleet and cannot appear as real route columns. Once feasibility is restored, the standard normalized objective is reinstated. This separation prevents a feasibility-recovery objective from being reported as a transportation result.

4.3 Exact pricing

Pricing is a shortest path problem with resource constraints (SPPRC) that searches for a feasible route $p \in \mathcal{P}(n)$ with $\bar{c}_p < -\varepsilon_{\text{rc}}$. A pricing decision cannot be reduced to choosing the next prospecting site. It must also choose one of the three declared path options, because the corresponding travel time, energy, risk, and shadow exposure affect different constraints and objective components. A proof-bearing labeling state consequently records the visited-task set, current node, depot availability, energy, load, cumulative shadow exposure, risk, science-weighted completion contribution, path signature, and reduced-cost terms. An open trip additionally carries the cumulative service-start offset of its last task and the current feasible interval for the trip's depot departure. Each task extension updates the offset and intersects that interval with the new task window. It never inserts waiting at the task. A depot return then selects the return option, tightens the interval by the mission-horizon bound in (3), closes the current trip, chooses its earliest feasible departure, reconstructs task completion times, introduces docking and recharge, and determines whether another task-disjoint trip can begin after any required depot wait.

The pricing model does not repeat the compact MILP. Instead, each defining constraint family in Section 3 is realized by a label invariant or transition:

Section 3 condition	Exact realization in proof-bearing pricing
Depot degrees, task in/out balance, and trip activation in (4a)	A label starts at the depot, every task extension appends exactly one incoming and one outgoing continuation, and a depot return closes the active trip

Section 3 condition	Exact realization in proof-bearing pricing
Route-level task uniqueness and elementarity in (4a)–(5)	The visited-task set rejects repeat visits both within and across trips of the same route
Time windows, no-wait arc equalities, and trip sequencing in (3) and (6a)	An open-trip label updates the cumulative offset and feasible depot-departure interval; trip closure uses its earliest feasible endpoint, and any delay before a later trip occurs only at the depot
Load, energy, and shadow limits in (6b)–(7)	Additive resource states are updated on every arc and task extension and are checked by exact feasibility pruning
Operating cost, risk, and weighted completion in (6b)	Additive objective and reduced-cost accumulators are updated from the same stored path and service quantities used by the master

Flow conservation is therefore a transition invariant rather than an additional numeric resource. The master-level exact task cover and fleet limit remain RMP rows and are not reimplemented as SPPRC resource constraints.

For a partial label L , let $[\underline{t}_L^0, \bar{t}_L^0]$ be the feasible depot-departure interval of its open trip. Resource infeasibility is a direct pruning condition. The proof-bearing no-task-wait test is

$$\begin{aligned}
\text{PRUNE}_{\text{res}}(L) = 1 \quad & \text{if } \underline{t}_L^0 > \bar{t}_L^0 + \epsilon_{\text{res}}, \\
& \text{or } Q_L > Q + \epsilon_{\text{res}}, \\
& \text{or } E_L > B + \epsilon_{\text{res}}, \\
& \text{or } H_L > H^{\max} + \epsilon_{\text{res}}, \\
& \text{or, after trip closure, } t_L^{\text{end}} > H^{\text{mis}} + \epsilon_{\text{res}}, \\
\text{PRUNE}_{\text{res}}(L) = 0 \quad & \text{otherwise.}
\end{aligned} \tag{16}$$

The last test is evaluated only after the return option and recharge terms are available. An empty departure interval means that no single depot departure can place every task in its window without task-site or en-route waiting. This pruning is an exact feasibility rejection under (3) and (6a)–(7), not a learned prediction.

A return to the depot completes a trip. The label sets $t_s^0 = \underline{t}_L^0$, reconstructs $t_{i_j, s}^{\text{cmp}} = t_s^0 + \delta_{j, s} + \sigma_{i_j}$, and then introduces docking and recharge. For the weighted-completion term, an open trip can store the affine expression $\sum_j w_{i_j}(\delta_{j, s} + \sigma_{i_j}) + t_s^0 \sum_j w_{i_j}$ until t_s^0 is fixed at closure. The route label can continue with a later task-disjoint trip; its new departure interval uses the previous trip end as the earliest depot availability. Terminal acceptance reconstructs the same operating-cost, risk, and weighted-completion quantities used in (10). This shared construction matters: a pricing objective that differs from the master objective can generate apparently attractive columns while invalidating the lower-bound argument.

Dominance removes a label only under rules proved compatible with every resource and reduced-cost term retained by the active state. Branch same/different-route decisions act as feasibility filters. Active deterministic cuts contribute both state information and dual terms. If a completion bound or dominance rule has not been proved for a nonempty branch or cut context, it is disabled for that context. In the current proof, completion-bound pruning admits branch restrictions but no active cut, whereas label dominance uses the explicit branch and cut guards below. The conservative behavior may increase work, but it does not silently enlarge the set of proof-bearing pruning operations.

More precisely, the current proof-bearing implementation compares route labels for dominance only when both are at the depot and under the same active branch-and-cut context; open-trip dominance is disabled. Let V_L be the route-level visited-task set, K_L the vector of active-cut resources, and a_L the depot-availability time of a depot label. Let $\text{BC}(L^1, L^2) = 1$ only when the active Ryan–Foster decisions preserve continuation: every branch-feasible continuation of L^2 is also branch feasible from L^1 . The no-wait comparability predicate is

$$\text{NW}(L^1, L^2) = \begin{cases} 1, & L^1, L^2 \text{ are at the depot, } a_{L^1} \leq a_{L^2} + \varepsilon_{\text{res}}, \\ & \emptyset \neq V_{L^1} \subseteq V_{L^2}; \\ 0, & \text{otherwise.} \end{cases}$$

Under these preconditions, L^1 may dominate and remove L^2 only when

$$L^1 \preceq L^2 \quad \text{only if} \quad \text{NW}(L^1, L^2) = 1 \wedge K_{L^1} = K_{L^2} \wedge \text{BC}(L^1, L^2) = 1 \\ \wedge \bar{c}(L^1) \leq \bar{c}(L^2) + \varepsilon_{\text{dom}}. \quad (17)$$

The nonempty-set condition prevents the initial empty-route label from removing a completed depot label. If L^1 has visited a subset of the tasks visited by L^2 , every continuation retained by $\text{BC}(L^1, L^2)$ remains available to L^1 ; equality of the active-cut state preserves future cut increments, and the reduced-cost test ensures that the corresponding completion from L^1 is no worse. Thus, a negative completion removed with L^2 has a no-worse negative counterpart from L^1 . The compatibility test is evaluated explicitly for active same-route branch decisions. Open-trip labels are never compared because a later task can force a retroactive common departure shift, and an earlier raw arrival alone does not preserve the feasible-departure interval when task-site waiting is forbidden.

The guarded completion bound uses only positive uncollected task-dual reward. Let $\bar{c}(L)$ be the partial reduced cost stored by label L , and let $\Pi_L^+ = \sum_{i \in V_L} \max(\pi_i, 0)$ be the positive task-dual reward already accumulated by that label. An optimistic reduced-cost lower bound and its pruning rule are

$$\underline{c}(L) = \bar{c}(L) - \max\left\{0, \sum_{i \in T} \max(\pi_i, 0) - \Pi_L^+\right\}, \quad \text{PRUNE}_{\text{cb}}(L) = \begin{cases} 1, & \text{if } \underline{c}(L) \geq -\varepsilon_{\text{rc}} + 10^{-12}, \\ 0, & \text{otherwise.} \end{cases} \quad (18)$$

All future operating-cost and resource increments are nonnegative by the input domain in Section 3.1, all completion-time weights are nonnegative, and the normalization references in (9) are strictly positive. Treating those future increments optimistically as zero in (18) can therefore only underestimate a completion's reduced cost. Branch restrictions only remove otherwise feasible continuations and introduce no additional negative reduced-cost term, so this lower bound remains optimistic under an active branch context. By contrast, an active cut can add a cut-dual contribution that is absent from (18). The proof-bearing implementation therefore enables this pruning only when the active-cut context is empty and forces it off whenever a cut is active.

Fast pricing and exact completion serve different purposes. The former may use dual ordering, learned priorities, active-support task sets, or limited path profiles to find negative columns quickly. A local failure to find one is not a proof. The proof-bearing true-dual completion pass must search an exact dominance-reduced representation of the complete multi-trip route space induced by the fixed graph and active context. The equal-travel-time same-endpoint substitution in Section 3.1 and Lemma 1 accounts for the limited set of path options removed before labeling. Only such a pass can support

$$\min_{p \in \mathcal{P}(n)} \bar{c}_p \geq -\varepsilon_{\text{rc}}. \quad (19)$$

If completion reaches a time, memory, label, or coverage limit, the node remains incomplete.

The frozen implementation realizes this rule in two equivalent forms. The fixed-route constructor intersects the task-induced departure intervals directly. The native pricing state stores the current common trip departure and its latest feasible value; when a later release time requires a delay, it shifts the common departure and every earlier completion contribution by the same amount rather than inserting task-site waiting. The compact formulation uses paired selected-arc temporal inequalities, so arrival and service start coincide. The timing policy, instance content, native engine, objective, and proof context are hash-bound in every accepted result row.

Figure 3 placeholder (FIG11, method evidence available): exact node workflow from RMP solve through ordered pricing, exact completion, deterministic separation, branching, and fail-closed incomplete termination.

4.4 Valid inequalities

The RMP is strengthened by one family of valid inequalities, namely subset-row inequalities defined over triples of tasks. The abbreviation SRI-3 is used hereafter: SRI denotes a subset-row inequality, and the suffix 3 indicates that the underlying task subset contains exactly three tasks. For any such triple, a route has coefficient one when it serves at least two tasks in the triple and coefficient zero otherwise. The resulting inequality allows at most one selected route to contain two or more tasks from that triple. It can eliminate fractional combinations of route columns, while exact task coverage ensures that no integer-feasible solution is removed.

These inequalities are separated only at the root node under a predefined deterministic policy. A root inequality that has been admitted remains a valid row in descendant RMPs, but descendants generate no new subset-row inequalities. The coefficient function is shared by the RMP, pricing, and reduced-cost audit. An active inequality therefore changes the RMP row set and the pricing dual context together. Its integer validity, pricing compatibility, context version, coefficient vector, lineage, and dual sign must all be auditable.

Let n_0 denote the root node and $\mathcal{S}_3 = \{S \subseteq \mathcal{T} : |S| = 3\}$ the family of task triples. For $S \in \mathcal{S}_3$, define $k_{Sp} = |\mathcal{T}_p \cap S| = \sum_{i \in S} a_{ip}$. The root-node divisor-two subset-row inequality is

$$\sum_{p \in \mathcal{P}(n_0)} \left\lfloor \frac{k_{Sp}}{2} \right\rfloor \lambda_p \leq 1, \quad a_{Sp}^{\text{SRI}} = \left\lfloor \frac{|\mathcal{T}_p \cap S|}{2} \right\rfloor. \quad (20)$$

Validity follows directly from exact task coverage. For any integer solution, $\sum_{p: \lambda_p=1} k_{Sp} = 3$, and hence $\sum_{p: \lambda_p=1} \lfloor k_{Sp}/2 \rfloor \leq \lfloor \sum_{p: \lambda_p=1} k_{Sp}/2 \rfloor = 1$. This proves validity for every integer solution of the task-partitioning master. Whether a valid root cut improves runtime is a separate empirical question.

At the root RMP solution λ^* , the deterministic separator enumerates the configured triples. Let $\varepsilon_{\text{sep}} = 10^{-6}$, and let $\nu_{\text{root}}(S)$ count the positive-support RMP columns that have a nonzero SRI-3 coefficient for S . Candidate construction, deterministic ordering, and cut harvesting are

$$\begin{aligned} \text{act}_{\text{root}}(S) &= \sum_{p \in \mathcal{P}'(n_0)} a_{Sp}^{\text{SRI}} \lambda_p^*, \\ \text{viol}_{\text{root}}(S) &= \text{act}_{\text{root}}(S) - 1, \\ \nu_{\text{root}}(S) &= \left| \left\{ p \in \mathcal{P}'(n_0) : \lambda_p^* > \varepsilon_{\text{int}}, a_{Sp}^{\text{SRI}} > 0 \right\} \right|, \\ \mathcal{C}_{\text{SRI3}}(n_0) &= \{S \in \mathcal{S}_3 : \text{viol}_{\text{root}}(S) > \varepsilon_{\text{sep}}\}, \\ \kappa_S &= (-\text{viol}_{\text{root}}(S), -\nu_{\text{root}}(S), \text{lex}(S)), \\ \mathcal{H}_{\text{SRI3}}(n_0) &= \text{Retains}_{\text{SRI3}}(\text{sort}_{\kappa}(\mathcal{C}_{\text{SRI3}}(n_0))). \end{aligned} \quad (21)$$

Thus, $\mathcal{C}_{\text{SRI3}}(n_0)$ contains every threshold-violating triple, whereas $\mathcal{H}_{\text{SRI3}}(n_0)$ contains only the cuts retained after the predefined separation round and lineage caps. The operator **Retains**_{SRI3} applies those deterministic caps after ordering by decreasing violation, decreasing positive-support count, and lexicographic task tuple. An active SRI-3 contributes exactly $-\gamma_S a_{Sp}^{\text{SRI}}$ to Eq. (13). The same coefficient remains active when a root cut is inherited by a descendant.

Cut-aware Phase I and proof binding prevent a stale or partial context from being used to accept a node bound. The proof record identifies the active-cut set and the dual/context fingerprint seen by proof-bearing pricing. Exact nonzero-dual projection may omit a cut from the pricing state only when its dual is exactly **0.0**, while preserving the full RMP and proof context; that projection is an implementation optimization, not a relaxation of cut validity.

Learning has no cut action in this paper. It neither proposes a subset, scores cut retention, chooses an activation round, nor deletes an active inequality. This exclusion keeps the experimental question focused on pricing and branching, and it avoids requiring a separate proof that a learned cut policy preserves separation validity and exact pricing compatibility.

4.5 Branching rule

Fractional task co-occurrence in the RMP solution supplies Ryan–Foster same/different-route candidates. For a selected pair (i, j) , one child requires i and j to occur in the same route and the other forbids that co-occurrence. The branch context is carried into route feasibility and proof-bearing pricing, so generated columns satisfy every decision on the path from the root to the node. The disjunction concerns whether two prospecting tasks share a complete one-rover multi-trip route; it does not prescribe their local path geometry or require them to occur in the same depot-to-depot trip.

Specifically, task-pair co-occurrence, fractionality, and the exact candidate set are

$$y_{ij} = \sum_{p \in \mathcal{P}'(n)} a_{ip} a_{jp} \lambda_p^*, \quad f_{ij} = \min\{y_{ij}, 1 - y_{ij}\}, \quad \mathcal{C}_{\text{RF}}(n) = \{(i, j) \in \mathcal{T} \times \mathcal{T} : i < j, \varepsilon_{\text{int}} < y_{ij} < 1 - \varepsilon_{\text{int}}\}. \quad (22)$$

The deterministic diagnostic order first maximizes f_{ij} , then minimizes the absolute difference between the numbers of currently available columns in the same-route and different-route children, and finally breaks ties by task identifiers. Learned branch guidance may replace this order over the unchanged set $\mathcal{C}_{\text{RF}}(n)$, but it does not alter y_{ij} , the fractionality test, or either child definition.

Candidate construction and tree completeness are separate obligations. Failure to find a fractional Ryan–Foster pair does not prove that the current solution is integral. Such a node can be resolved only by an exact alternative disjunction or by a separately derived proof that the fractional representation corresponds to an integral scheduling decision. Because neither mechanism is currently available in this case, the node remains unresolved and no exact tree-level conclusion is drawn. The learned branch ranker enters only after exact logic has formed a valid candidate set and cannot alter this requirement.

4.6 Learning guidance

The learning interface has three parts. A graph-state representation exposes only the solver information required for ranking; typed hints bind every prediction to its exact context; and separate pricing and branching interfaces restrict learning to work order. This design retains the lunar task, path, and resource structure needed to distinguish expensive pricing choices while preventing a prediction from changing the transportation model. The following subsections define these parts in the order in which they enter the exact framework.

4.6.1 Graph representation

The planned guidance representation preserves the directed task graph and the distinction among path alternatives. Task nodes contain depot/task type, location, operation mode, science weight, demand, service quantities, time windows, and shadow/thermal indicators. Directed pair and path-option features represent relative geometry, travel time, energy, risk, distance, and shadow exposure. Solver-state features provide the current dual, branch, cut, candidate, and workload context. Graph neural networks are appropriate representational tools for such structured solver states [C003; C008], but the representation alone is not evidence of an effective policy.

The exact interface accepts typed hints rather than a model object. Each hint binds a candidate identifier and signature to a priority, uncertainty, finite-delay budget, source, branch/cut/path-option context signatures, reduced-cost fingerprint, model identifier, and feature-schema version. A context mismatch, an unavailable model, or an out-of-distribution trigger reverts the decision to shadow-only or deterministic behavior. The exact BPC does not import a training framework or checkpoint loader.

[[TBD-M002: Insert the frozen pricing-guidance checkpoint hash, feature-schema version, architecture, training target, loss, model-selection rule, calibration fields, and inference environment. Leave all values empty until the model package exists.]]

[[TBD-M003: Insert the frozen branch-ranking checkpoint, candidate-label construction, valid-candidate logging schema, and branch inference environment. Leave all values empty until the model package exists.]]

The current implementation contains the typed safety interface and a deterministic shadow-only execution path. It does not establish that a trained GAT has been produced, selected, or evaluated. The placeholders above are intentionally located inside the method section so that later implementation details can be inserted without changing the responsibility boundary or the order of the algorithm.

4.6.2 Pricing guidance

Pricing guidance ranks work items such as task expansions, path-option expansions, worker seeds, candidate task sets, or expensive pricing calls. The desired mechanism is earlier discovery of useful negative-reduced-cost routes, not a change to the set of routes that may eventually be searched. This role parallels learned and selective pricing research [C002; C009; C059], while the present interface makes exact completion mandatory before a proof-bearing node bound is accepted.

A finite delay is permitted only if the associated pricing obligation remains explicit. Every delayed item enters $\mathcal{D}_n$ immediately with its context binding and with unreconstructed reduced cost. A recheck removes the item only after it is shown nonnegative, processed as a true-negative candidate, or covered by a complete true-dual repricing pass. Before a no-negative-column proof or node bound is established, every item must therefore be resolved. Thus, the learned order may affect when work is performed but cannot permanently discard a column that the exact algorithm requires.

Let $\mathcal{D}_n$ contain unreleased delayed candidates, with $\bar{c}_d = \perp$ when the true reduced cost has not yet been reconstructed. Let b_n^{delay} indicate whether this set contains an unresolved item that could invalidate pricing closure:

$$b_n^{\text{delay}} = \begin{cases} 1, & \text{if some } d \in \mathcal{D}_n \text{ has } \bar{c}_d = \perp \text{ or } \bar{c}_d < -\varepsilon_{\text{rc}}, \\ 0, & \text{otherwise,} \end{cases} \quad (23)$$

$$\text{a proof-bearing conclusion at node } n \Rightarrow \mathcal{D}_n = \emptyset \wedge b_n^{\text{delay}} = 0.$$

Operationally, deferred items are released before a proof attempt and are reprocessed or subsumed by exhaustive true-dual completion. Merely clearing the set without performing that exact work would not satisfy (23).

Algorithm 2. Pricing guidance with exact completion and deferred-work control

Require: node n ; true dual tuple (π, μ, γ) ; branch context $\mathcal{B}(n)$; deterministic-cut context $\mathcal{H}(n)$; pricing work set $\mathcal{W}(n)$; unresolved deferred-pricing set $\mathcal{D}$; reduced-cost tolerance ε_{rc} ; optional typed pricing hints.

Ensure: one of three outcomes: an audited set of exact-addable negative-reduced-cost routes; a proof that no such route exists in $\mathcal{P}(n)$; or termination without a pricing-closure proof.

Line	Procedure
1	Validate every hint against the candidate signature, feature schema, model identifier, dual fingerprint, and branch/cut/path-option context.
2	Reject unavailable, mismatched, or out-of-distribution hints and use the deterministic work order for their items.
3	Reorder $\mathcal{W}(n)$ by accepted priorities without deleting any work item; enter each delayed item in $\mathcal{D}$ with $\bar{c}_d = \perp$ and enforce every finite-delay budget.
4	Run ordered fast pricing and reconstruct each returned route's true reduced cost using (13).
5	if an exact-feasible, exact-addable route has $\bar{c}_p < -\varepsilon_{\text{rc}}$ then return the audited batch of negative-reduced-cost routes.
6	Recheck every due item in $\mathcal{D}$ by true-reduced-cost reconstruction and exact processing; leave any unresolved item recorded.

Line	Procedure
7	Run the verified exact SPPRC completion over $\mathcal{P}(\mathbf{n})$ with the true duals and the complete active context; clear a remaining item from $\mathcal{D}$ only when this pass covers its bound context.
8	if exact completion finds an addable negative-reduced-cost route then return the audited batch containing that route.
9	if exact completion exhausts the frontier, all reduced-cost and context audits pass, and $\mathcal{D} = \emptyset$ then conclude that no negative-reduced-cost route exists in $\mathcal{P}(\mathbf{n})$.
10	otherwise terminate without a pricing-closure proof.

Lines 1–3 convert learned output into a permutation with bounded delay rather than a filter. Lines 4–6 retain true-reduced-cost admission and deferred-work accounting. Lines 7–10 form the exact completion tail: only exhaustive proof-bearing pricing with no unresolved deferred-pricing obligation can prove the absence of a negative-reduced-cost route. The algorithm defines the permitted learned interface; it does not imply that the TBD pricing checkpoint in M002 has been trained or evaluated.

4.6.3 Branch guidance

Branch guidance ranks an exact-valid candidate set and determines which candidates receive expensive evaluation first. It may approximate a strong branching preference or a downstream pricing-pressure signal, as suggested by learning-to-branch precedents [C001; C003]. The selected candidate still passes the exact branch constructor, and its two children retain the complete same/different-route partition.

When the score is absent, rejected by a context check, or too uncertain, the deterministic candidate rule is used. When no Ryan–Foster pair is available, the exact fallback remains unchanged. Branch ranking is secondary in the paper because it acts less frequently than pricing and because its incremental value must be measured against a variant that already contains pricing guidance.

Algorithm 3. Branch-candidate ranking over exact-valid Ryan–Foster pairs

Require: fractional RMP solution λ^* ; node context $\mathcal{B}(\mathbf{n})$; exact Ryan–Foster candidate constructor; deterministic candidate/evaluation budget; deterministic candidate rule; optional typed branch-ranking hints.

Ensure: two exact child contexts or an explicit exact fallback/incomplete outcome.

Line	Procedure
1	Construct the deterministically bounded set $\mathcal{C}_{\text{RF}}(\mathbf{n})$ of fractional, exact-valid Ryan–Foster task pairs from λ^* .
2	if $\mathcal{C}_{\text{RF}}(\mathbf{n}) = \emptyset$ then invoke an available exact representative/variable fallback or require an aggregation proof; if neither exists, return an explicit incomplete outcome rather than declaring integrality.
3	Validate every branch hint against a member of $\mathcal{C}_{\text{RF}}(\mathbf{n})$ and the current node, candidate-set, branch, and model-schema signatures.
4	Reorder $\mathcal{C}_{\text{RF}}(\mathbf{n})$ by accepted hints; if a hint is absent, invalid, or too uncertain, use the deterministic order.
5	Verify that the ordered set contains exactly the candidates constructed on Line 1; no candidate may be added or removed by learning.
6	Evaluate candidates in that order under the frozen exact branch-selection policy and select an exact-valid pair (i, j) .
7	Construct $\mathcal{B}^{\text{same}}(\mathbf{n}; i, j)$ and $\mathcal{B}^{\text{different}}(\mathbf{n}; i, j)$ with the exact branch constructor.
8	Validate that generated routes in each child satisfy its full inherited branch context.
9	if both child contexts pass validation then return the two children; else use the next exact-valid candidate or deterministic fallback.

Lines 1–2 keep candidate construction and the no-pair proof boundary on the exact path. Lines 3–5 permit learning to change only the inspection order. Lines 6–9 leave candidate evaluation and same/different-route child construction exact and otherwise fail closed. This component remains design-only until M003 and the L2 experiment block are frozen.

4.7 Exactness proof

This section establishes the exactness of the complete BPC algorithm, not only the safety of its learning interface. The result is conditional on the declared fixed logical-path solution space and on completion of every proof-bearing operation. It proves the correctness of a returned exact conclusion; it does not assert that every instance must reach such a conclusion within finite computational limits.

For the revised formulation, this is both a mathematical and an implementation contract. A solver may invoke the theorem only after an audit verifies that its column constructor and exhaustive pricing procedure implement the depot-departure logic, no-wait transitions, restricted path-option preprocessing, and guarded label dominance stated below. The frozen revised implementation satisfies that activation condition; predecessor wait-permitted runs do not.

Unless stated otherwise, Lemmas 1–5 and Theorem 1 use exact arithmetic and set all numerical comparison tolerances to zero. The positive tolerances used by the executable solver are qualified separately after the proof.

For a node $\mathbf{n}$, let $\mathcal{F}(\mathbf{n})$ contain the integer vectors that satisfy the master problem (12) under branch context $\mathcal{B}(\mathbf{n})$ and active deterministic-cut context $\mathcal{H}(\mathbf{n})$. The node optimum is

$$z^*(\mathbf{n}) = \min_{\lambda \in \mathcal{F}(\mathbf{n})} \left\{ \sum_{p \in \mathcal{P}(\mathbf{n})} c_p \lambda_p \right\}, \quad (24)$$

with $z^*(\mathbf{n}) = +\infty$ when $\mathcal{F}(\mathbf{n})$ is empty.

Lemma 1 (completeness of the canonical route representation). Suppose that the path travel, energy, risk, and shadow quantities are independent of absolute departure time; the stored resource and cost quantities and all completion-time weights are nonnegative; waiting is permitted only at the depot and has no negative cost; recharge duration depends only on the energy used; and path-option dominance is restricted to the equal-travel-time rule in Section 3.1. For every feasible multi-trip route allowed by (4a)–(7), there is a canonical route with the same task sequence, the same task-site and en-route no-wait policy, no larger resource consumption, and no larger cost (10). Consequently, an optimal solution over the fixed logical-path solution space has a representative in the route set searched by any exact pricing procedure that implements Eqs. (3), (6a), and (16)–(19).

Proof. Fix one feasible multi-trip route, including its task sequences and path-option choices. If a selected path option was removed under Section 3.1, replace it by its retained equal-travel-time counterpart. Every task time within the affected trip remains unchanged, while energy, risk, distance, and shadow exposure weakly decrease. Keeping the current and subsequent trip departures unchanged, any reduction in recharge duration is absorbed as additional depot waiting. Hence all task arrival and completion times can be preserved, while depot availability weakly improves. The replacement preserves all task-cover, cut, and branch coefficients and cannot violate a resource or temporal constraint.

Consider trip $\mathbf{s}$ after all earlier trips have been fixed canonically, and let $\mathbf{a}_{\mathbf{s}}$ be the resulting earliest depot-availability time. For the fixed task sequence and path choices, the offsets $\delta_{j,\mathbf{s}}$ and duration $\chi_{\mathbf{s}}$ in (3) are constants. No waiting is allowed after departure, so task i_j meets its time window exactly when

$$r_{i_j} - \delta_{j,\mathbf{s}} \leq t_{\mathbf{s}}^0 \leq D_{i_j} - \sigma_{i_j} - \delta_{j,\mathbf{s}}.$$

The depot-availability and mission-horizon requirements add $t_s^0 \geq a_s$ and $t_s^0 \leq H^{\text{mis}} - \chi_s$. Their intersection is precisely $[\underline{t}_s^0, \bar{t}_s^0]$ in (3). The original trip is feasible, so this interval is nonempty. Choosing $t_s^0 = \underline{t}_s^0$ therefore preserves every release time, due time, return, recharge, and horizon constraint without creating task-site or en-route waiting. It also gives the earliest completion of every task in that fixed trip.

For $s = 1$, $a_1 = 0$. Suppose trips $1, \dots, s-1$ have been replaced by their canonical schedules. Their final depot availability is no later than under any later feasible choice. If the next trip would otherwise need to start later, the rover can wait at the depot. More specifically, the departure used by the original feasible trip remains in the revised interval because its lower availability bound has only decreased; hence the interval for trip s is not lost. Induction over the ordered trips gives a feasible canonical multi-trip route. Load and path resources are unchanged apart from the weak improvements caused by equal-travel-time option substitution. Depot waiting adds no trip resource or operating cost, and nonnegative science weights make the earliest feasible completion vector weakly preferable. Finally, the numbers of tasks, retained path options, and trip slots are finite. The canonical route set is therefore finite and contains an optimal representative. $\square$

Lemma 2 (equivalence of route selection and feasible fleet schedules). At the root node, every vector in $\mathcal{F}(n_0)$ defines a member of $\Omega(\mathcal{I})$, and every fleet schedule in $\Omega(\mathcal{I})$ has a canonical vector in $\mathcal{F}(n_0)$ with no greater objective value. Hence the two representations have the same optimum.

Proof. Each selected column is a feasible one-rover multi-trip route by the definition of $\mathcal{P}(n_0)$. The task-cover equalities in (12) assign every task to exactly one selected route, while the fleet inequality provides a distinct rover for every selected route. Hence a vector in $\mathcal{F}(n_0)$ gives a feasible fleet schedule. Conversely, decompose any feasible fleet schedule into its nonempty one-rover multi-trip routes. Lemma 1 supplies an equal-or-better canonical representative for every route. Selecting the corresponding columns satisfies exact task cover and the fleet limit. The validity derivation following (20) ensures that the selection also satisfies every active root SRI-3 row. Additivity in (8)–(10) gives the same, or a weakly smaller, objective; an optimal fleet schedule can therefore be represented without loss. $\square$

Lemma 3 (exact node-LP closure). Assume that the RMP at node n is solved to LP optimality, Eq. (13) is used by the RMP, pricing, and reduced-cost audit, and proof-bearing exact completion explores all retained labels not removed by a proved extension-preserving feasibility, dominance, or completion-bound rule under the current branch-and-cut context. If exact completion proves nonnegative reduced cost over this dominance-reduced representation, then $\bar{c}_p \geq 0$ for every $p \in \mathcal{P}(n)$, and the RMP optimum equals the LP-relaxation optimum over all columns in $\mathcal{P}(n)$:

$$z_{\text{RMP}}(n) = z_{\text{LP}}(n) \leq z^*(n). \quad (25)$$

Proof. Let (π, μ, γ) be the audited optimal RMP dual solution. An equal-travel-time same-endpoint path substitution leaves task, fleet, cut, branch, and timing coefficients unchanged and weakly decreases c_p ; it therefore weakly decreases the reduced cost in (13). By Lemma 1, a negative-reduced-cost route using a safely removed option would have a retained feasible counterpart with no greater reduced cost. Exact completion and (19) thus imply, in exact arithmetic, that $\bar{c}_p \geq 0$ for every $p \in \mathcal{P}(n)$. Hence the RMP dual vector satisfies the dual constraint associated with every column of the full node master, including columns absent from the RMP. It is therefore feasible for the full node dual. Strong LP duality gives $z_{\text{RMP}}(n) \leq z_{\text{LP}}(n)$. Because the RMP contains only a subset of the full primal columns and the problem is a minimization problem, $z_{\text{LP}}(n) \leq z_{\text{RMP}}(n)$. Equality follows. Finally, the LP relaxation is a relaxation of the integer master, so $z_{\text{LP}}(n) \leq z^*(n)$. If the RMP is initially infeasible, the same argument applies to Phase I: a positive full Phase-I optimum after exhaustive Phase-I pricing proves that no zero-artificial LP solution, and therefore no integer solution in $\mathcal{F}(n)$, exists. $\square$

The extension-preserving qualification in Lemma 3 is essential. Resource pruning in (16) removes only labels that already violate a defining constraint, including an empty feasible depot-departure interval. The equal-travel-time path-option rule is covered by the substitution argument in Lemma 1. Label dominance in (17) may be used by a proof-bearing completion pass only when each removed label has a retained label capable of reproducing every feasible continuation with no larger reduced cost. The implemented rule therefore compares only depot labels, excludes the empty initial visited set, and requires the active-cut and branch-continuation guards in (17); open-trip dominance is disabled. The completion bound in (18) may be used only in its proved context. Any unsupported pruning rule must be disabled. If a time, memory, label, or frontier-coverage limit prevents exhaustive completion, the node remains unresolved and Lemma 3 does not apply.

Lemma 4 (preservation under cuts and branching). Admitting a root-node SRI-3 cut defined by (20) does not remove an integer solution of (12). For an exact-valid Ryan–Foster pair (i, j) at node n , the same-route and different-route children satisfy

$$\begin{aligned}\mathcal{F}(n) &= \mathcal{F}(n_{ij}^{\text{same}}) \cup \mathcal{F}(n_{ij}^{\text{different}}), \\ \mathcal{F}(n_{ij}^{\text{same}}) \cap \mathcal{F}(n_{ij}^{\text{different}}) &= \emptyset.\end{aligned}\tag{26}$$

Proof. The SRI-3 validity derivation following (20) shows that the inequality holds for every integer exact-cover solution. Adding or inheriting such a row can remove fractional LP solutions but not a member of $\mathcal{F}(n)$. Now fix an integer solution at node n . Exact task cover implies that tasks i and j are either served by the same selected route or by two different selected routes; exactly one case holds. The same-route child rejects columns containing only one member of the pair, whereas the different-route child rejects columns containing both. The integer solution therefore belongs to exactly one child, which proves (26). Learning may change which exact-valid pair is considered first, but it cannot change either child definition. If no valid pair or exact alternative disjunction is available for a fractional node, the node remains incomplete and cannot be closed as integral. $\square$

Lemma 5 (preservation under learning guidance). Suppose accepted learning outputs only permute pricing work and exact-valid branch candidates, every finite delay is released or covered by exhaustive true-dual repricing before a proof-bearing event, and guidance cannot modify column admission, cuts, branch contexts, bounds, pruning, or proof records. Then guidance does not change any feasible set $\mathcal{F}(n)$, any route cost c_p , or the validity of Lemmas 1–4.

Proof. A permutation changes only the time at which an unchanged work item is processed. Finite delay has the same property once every delayed item is processed or subsumed by exhaustive exact completion. The necessary condition in (23) prevents a proof while any deferred-pricing obligation is unresolved. Exact column admission still uses (13) and (15), cuts remain those proved valid in Lemma 4, and branching still uses the two exact child definitions in (26). Guidance therefore changes an execution trace, and may change the selected valid branch pair, without changing the mathematical alternatives covered by the trace. $\square$

Theorem 1 (conditional exactness of the complete BPC algorithm). Consider a frozen instance $\mathcal{I}$ and the fixed logical-path solution space $\Omega(\mathcal{I})$. Assume the conditions of Lemmas 1–5 and the pruning rule in (14). Suppose the search terminates with an exactly feasible incumbent and a finite closed branch-price-and-cut tree in which every leaf has been resolved by integrality, exact Phase-I infeasibility, or valid bound pruning; every processed node not proved infeasible has a lower bound satisfying Lemma 3; and no pricing, audit, branching, or deferred-work obligation remains unresolved. Then the incumbent satisfies

$$z^{\text{inc}} = z^*(n_0).\tag{27}$$

A node reported as exactly infeasible has $\mathcal{F}(n) = \emptyset$. If any required RMP solve, pricing completion, audit, branch disjunction, or tree operation remains incomplete, the algorithm does not produce either conclusion.

Proof. At the root, Lemmas 1 and 2 show that $\mathcal{F}(\mathbf{n}_0)$ represents the complete feasible fleet-schedule set within $\Omega(\mathcal{I})$. Consider any processed node. Lemma 3 provides a valid node lower bound after exact pricing closure. If the closed RMP has an integral optimal solution, that solution is also feasible for the full node LP, so its value equals the node integer optimum and it is a valid incumbent candidate. If (14) prunes the node, the valid lower bound is no smaller than the incumbent value, so the node cannot contain a better integer solution. Exact Phase-I infeasibility makes the node feasible set empty. Otherwise Lemma 4 partitions the node feasible set into two disjoint children without loss.

Induction over the generated tree therefore shows that the union of the feasible sets of the active leaves, the already accepted integer leaves, and the validly pruned leaves accounts for every solution in $\mathcal{F}(\mathbf{n}_0)$. A global optimality conclusion is valid only if an exactly feasible incumbent exists; no node remains open or unresolved; every processed node not proved infeasible has a lower bound satisfying Lemma 3; every leaf has been resolved by integrality, exact infeasibility, or valid bound pruning; all proof obligations have been verified; and no delayed pricing work remains unresolved. Under these conditions, no unaccounted solution at the root can have value below z^{inc} , while the incumbent itself belongs to $\mathcal{F}(\mathbf{n}_0)$. Equation (27) follows. Lemma 5 shows that accepted learning orders do not alter this argument. $\square$

The theorem proves the soundness of an exact conclusion, not unconditional finite-time completion. In particular, the present Ryan–Foster implementation terminates without an exact conclusion when a fractional node has no valid pair and no proved alternative disjunction. Tree-depth, node, time, label, memory, and coverage limits are treated according to the same conservative rule. These outcomes preserve soundness but do not establish algorithmic completeness for every instance.

The proof above is stated in exact arithmetic with zero comparison tolerances. The implementation uses ϵ_{rc} , ϵ_{bnd} , ϵ_{int} , and solver feasibility tolerances, so a computationally reported conclusion is tolerance-qualified rather than the exact-arithmetic equality itself. This numerical qualification is distinct from the combinatorial exactness established by Lemmas 1–5. It is also distinct from physical-model validity: Eq. (27) applies only to the frozen task graph and declared path alternatives.

The do-no-harm audit operationalizes the theorem’s conditions by checking incumbent feasibility, consistency of the objective and proof scope, reduced-cost reconstruction, exact-pricing coverage, active branch and cut contexts, outstanding pricing obligations, preservation of the branch candidate set, the absence of unresolved nodes, and the validity of the proof records. Any failed check blocks both performance interpretation and a tree-level exact conclusion.

5. Experimental Design

This section fixes the evaluation logic before interpreting learning effects. It first states the research questions and data provenance, then defines the learning splits and compared variants, and finally places exactness gates before workload, runtime, and generalization analyses.

5.1 Research questions

The evaluation addresses five questions:

1. **RQ1 — Exactness preservation.** Do L1 and L2 reproduce the exact results and proof conditions of L0?
2. **RQ2 — Pricing guidance.** Does learned pricing order change end-to-end effort after inference overhead and exact fallback are counted?
3. **RQ3 — Branch guidance.** Does branch-candidate ranking add value beyond pricing guidance alone?
4. **RQ4 — Generalization and fallback.** How do the policies behave on held-out maps, seeds, or scales, including out-of-distribution cases?
5. **RQ5 — Seasonal operating phase.** When task locations, rover parameters, mission horizons, and normalization references are held fixed, how do four south-polar seasonal operating phases affect path availability, route feasibility, normalized science-weighted completion time, reporting-only makespan, resource use, and the selected fleet schedule?

These questions are neutral; no direction is assumed before the corresponding frozen results exist.

5.2 Benchmark instances

The benchmark manifest contains 120 accepted instances, with 20 instances at each task scale **5, 10, 20, 30, 50, and 100**. All instances use the same **50 km × 50 km** base map and forward-looking mobility scenario; scale changes task density rather than spatial extent. Each directed task-graph edge follows the recorded three-path policy, and the instances contain detect, drill, and sample operations. Fleet size and mission horizon increase with scale according to the manifest. Specifically, task scales **5, 10, 20, 30, 50, and 100** use fleet sizes **1, 2, 3, 4, 5, and 8**, and mission horizons **960, 960, 1680, 1680, 3000, and 4560** min, respectively. Thus, the planning windows range from **16 to 76** h rather than one lunation. The corpus count is not an exact-solve count: the frozen exact baseline reported in Section 6 covers the 80 instances at scales 5–30, whereas scales 50 and 100 currently have bounded fail-closed evidence only.

The current route constructor, compact formulations, Python pricing path, and native exact pricer implement the no-task-wait rule. Their shared policy is bound to generated instances, cached pricing graphs, solver payloads, and proof records. A completed 80-instance run at scales 5–30 supplies new exact evidence for this formulation. Earlier experiments that permitted early-arrival waiting remain a separate historical evidence class. Their runtime, pricing workload, cut-effect, state-size, and scalability measurements are not transferred to the revised model.

The map-source catalog records locally available LOLA-derived slope, roughness, permanently shadowed region, and illumination-related inputs and their roles. Derived resource and risk layers are benchmark inputs rather than direct observations of ice abundance. Figure 1 and the final data table must retain that provenance distinction.

The multi-epoch analysis is designed as a paired instance study rather than a departure-time-dependent extension of the solver. Each paired family must use the same task locations, service data, rover parameters, fleet limit, mission horizon, and objective-normalization references. Only the mission epoch, hourly illumination samples, resulting mission-window environmental summary, generated path records, and quantities derived from those records may

change. The planned design starts at the southern vernal equinox and uses **12** epoch anchors uniformly spaced over the **346.6**-Earth-day draconic year, approximately **28.9** Earth days apart. It evaluates the scale-dependent **16–76** h mission window from each anchor with one-hour environmental sampling. The near-lunation interval is the spacing between independent environmental scenarios, not the route duration. A common normalization basis is required because separately rescaling every epoch would make normalized objective values incomparable.

The **12** anchors are grouped into four standard solar-declination phases at the lunar south pole, with three consecutive anchors in each phase. An anchor on a boundary is assigned to the phase that begins at that boundary.

Phase	Interval	Environmental tendency at the south pole	Anchor indices
South-polar spring	Southern vernal equinox to southern summer solstice	Solar elevation increases toward its seasonal maximum	0–2
South-polar summer	Southern summer solstice to southern autumnal equinox	Solar elevation decreases from its seasonal maximum	3–5
South-polar autumn	Southern autumnal equinox to southern winter solstice	Solar elevation decreases toward its seasonal minimum	6–8
South-polar winter	Southern winter solstice to the next southern vernal equinox	Solar elevation increases from its seasonal minimum	9–11

Kloos et al. show that polar seasonal shadow changes with topography and time of year over the draconic cycle [C063]. Wei et al. further use a season-conditioned southern-summer illumination analysis near Shackleton crater and relate seasonal polar illumination to landing-site selection, solar-power use, and traverse design [C064]. Together, these studies support treating the operating phase as a controlled environmental factor rather than using one annual average for every mission period. The four equinox-to-solstice groups remain a balanced experimental stratification of the present study, not a phase classification or ranking prescribed by either source. The phase labels organize the environmental comparisons; they do not replace the hourly, terrain-specific preprocessing used to construct each fixed instance. For each paired task family and phase, the arithmetic mean is taken over the three anchors only when all three rows are exact and feasible. Exact infeasibility is reported as a separate phase outcome, and a family-phase containing an incomplete row is withheld from exact completion-time ranking. Pairwise phase contrasts are then calculated on the common exact-feasible families, with paired uncertainty intervals, so the three anchors of one task family are not treated as independent replications. The primary temporal measure is normalized science-weighted completion time because it is the completion-time component of the declared objective. Makespan is reported separately as a secondary operational measure and is not added to the objective. If the two measures give different phase orderings, the result is reported as a trade-off rather than reduced to one fastest phase.

[[TBD-M006: Insert the frozen multi-epoch instance manifest, including a southern-vernal-equinox reference, 12 epoch anchors, their approximately 28.9-Earth-day spacing, the four three-anchor phase labels, the scale-dependent mission windows, one-hour illumination samples, the declared window-aggregation rule, illumination-map provenance, path-generation configuration and hashes, paired task sets, common normalization references, instance counts, exact/incomplete status, family-level phase summaries, paired phase contrasts, uncertainty intervals, and row paths. The package must also quantify within-window environmental variation and state whether the chosen aggregation is representative or conservative. Until this package exists, the frozen results in Section 6 remain single-environment-instance evidence and do not establish a seasonal operating-phase effect.]]

5.3 Data partitioning

[[TBD-M001: Insert the frozen sample inventory and train/validation/test split. Required fields are corpus and row hashes; map, seed-family and scale groups; target construction; excluded overlaps; leakage tests; and the held-out unit. No random-row split may serve as the main generalization result.]]

The split placeholder is positioned before variant and training details because every learned comparison depends on it. Once M001 exists, model selection must use only its training and validation partitions, and all held-out claims must name the untouched map, seed, or scale group.

5.4 Compared methods

Three exact variants isolate the two learned actions:

- **L0 — Exact control:** exact BPC without learned ordering.
- **L1 — Pricing guidance:** L0 plus learned pricing order, with the same RMP, objective, verified no-task-wait exact completion, deterministic cuts, branch logic, and proof records.
- **L2 — Pricing and branch guidance:** L1 plus branch-candidate ranking over the same exact-valid candidate sets.

No variant uses learning to generate, select, activate, retain, or delete cuts.

A deterministic pricing schedule is the primary algorithmic comparator. Selective pricing is a relevant literature baseline because it also allocates effort across weaker and stronger pricing stages [C059]. Any implemented comparator must be frozen with the same instances, solver budgets, exact fallback, and objective; naming a comparator in this design does not imply that its project implementation already exists.

5.5 Implementation details

The no-task-wait implementation is bound by an immutable manifest covering the route constructor, compact formulations, Python and native pricing paths, configuration, native binary, tests, and instance contents. For a fixed trip, the constructor computes a common feasible-departure interval. The native state represents its lower endpoint by the current trip departure and retains the latest feasible departure explicitly; a later task release shifts the whole trip and its previously accumulated weighted-completion contribution. Service begins on arrival, and no task-site or en-route waiting variable is created. Path-option preprocessing requires equal travel time within the recorded proof-bearing native tolerance, active-trip dominance is disabled, and depot subset dominance is admitted only for a nonempty visited set with identical cut state and continuation-preserving branch compatibility. The optional completion bound is disabled in the frozen 80-instance baseline; therefore, its branch-context scope does not affect the results in Section 6.1.

Targeted tests verify policy/hash binding, rejection of a route that would be feasible only through task waiting, and agreement between direct pricing and the compact model. The native test suite independently checks the same timing semantics. The frozen 80-instance package contains one fresh strict-cold runtime per instance, records no solver resume or external pool, binds a single source bundle and native engine, and passes its independent freeze verification. These artifacts activate the revised-model baseline in Section 6.1. New runs remain necessary before any historical cut-effect, state-refinement, or scale-50/100 resource measurement is attributed to the no-task-wait model.

[[TBD-M002: Insert pricing model input dimensions, architecture, targets, training loss, optimizer, selection criterion, checkpoint hash, calibration, and inference hardware/software.]]

[[TBD-M003: Insert branch model target, candidate-set construction, architecture, loss, checkpoint hash, and inference configuration.]]

No provisional hyperparameter is inserted in this working draft. The final training subsection must also report failed runs or model-selection changes that affect the evidence lineage.

5.6 Exactness verification

Safety is evaluated before workload. The frozen implementation baseline has passed the implementation-verification gate for the no-task-wait formulation. Before learning variants are compared, each variant must bind the same timing policy, instance content, objective, pricing semantics, and proof context, and no task extension may insert a task-site or en-route delay. The zero-tolerance endpoints then cover four failure groups: (i) an unsupported exact conclusion, an unsupported claim that no negative-reduced-cost route exists, or invalid node pruning; (ii) objective or reduced-cost mismatches; (iii) unresolved deferred-pricing obligations or permanent loss of a required negative column; and (iv) loss of an exact-valid branch candidate, unsupported branch/cut context, or missing proof lineage. A single safety failure prevents a performance claim for the affected comparison.

The safety table is intentionally separate from runtime. It should report each variant's exact rows, objective and reduced-cost mismatches, false proof or pruning decisions, unresolved deferred-pricing obligations, and candidate-set mismatches. This prevents a faster but mathematically different run from appearing favorable.

5.7 Performance evaluation

End-to-end wall time is paired with mechanism-level metrics: RMP and pricing time, generated and extended labels, pricing calls by mode, columns found and admitted, exact-completion and final-judge calls, branch nodes, branch candidate evaluations, child pricing workload, inference time, fallback frequency, and peak memory. Incomplete runs retain their terminal class and denominator. Reporting only completed or favorable cases is not permitted.

The learning study will use strict-cold paired schedules, repeated runs, fixed hardware and resource limits, recorded AB/BA order, and uncertainty intervals over the predefined comparison unit. Exact repetition counts, seeds, time limits, memory limits, build hashes, and checkpoint hashes remain empty until the experiment manifest is frozen.

[[TBD-M004: Insert L0/L1/L2 paired run design, repetition counts, schedules, hardware, limits, build/config hashes, row paths, and summary hashes.]]

5.8 Generalization evaluation

The held-out study must identify the unit excluded from training, define the OOD criterion, record calibration or uncertainty, and report deterministic fallback. Exact closure is evaluated independently of policy confidence, because OOD fallback may remove any learned workload benefit while preserving the exact solution.

[[TBD-M005: Insert exact-equivalence audit, inference overhead, fallback frequency, OOD definition, held-out rows, and frozen summaries.]]

6. Computational Results

The results are ordered by evidence maturity. Section 6.1 reports the frozen exact baseline for the no-task-wait formulation. Sections 6.2 and 6.3 retain historical deterministic-cut and exact-state evidence under the predecessor wait-permitted timing rule. Section 6.4 reserves the pending learning evaluation, Section 6.5 reserves the paired seasonal operating-phase analysis, and Section 6.6 states the current historical resource-limited boundary.

6.1 Exact baseline

The frozen implementation baseline uses the no-task-wait timing rule and deterministic root-only SRI-3. All 80 instances at scales 5, 10, 20, and 30 reached exact termination and passed the recorded correctness and integrity checks under the bound source bundle, configuration, native engine, and instance hashes. Each scale contains 20 instances and one fresh strict-cold run per instance; solver resume and external column pools were disabled. This result does not extend to scales 50 or 100 and does not measure a learning effect.

Table 1. Frozen strict-cold exact baseline under the no-task-wait timing rule and root-only SRI-3. Times are descriptive for the bound build and 20 instances per scale.

Tasks	Exact / total	Mean time (s)	Median time (s)	Maximum time (s)
5	20 / 20	0.442	0.446	0.479
10	20 / 20	1.461	0.988	5.178
20	20 / 20	39.820	17.106	201.125
30	20 / 20	177.358	77.281	716.426

Runtime increases sharply across these recorded scales, and the distributions are skewed at scales 20 and 30 because their means exceed their medians. The table is descriptive. It does not identify an asymptotic law or prove that one solver component alone causes the growth. The predecessor wait-permitted control is not used as a speed comparator because prohibiting task waiting changes the feasible set and, for most instances, the optimum. In the paired instance audit, the revised objective never decreased: it was unchanged for 17 scale-5 instances and increased for the remaining 63 instances. This monotonicity is a model-consistency check, not an algorithmic performance claim.

Figure 4 placeholder (FIG12, evidence available): distribution-aware strict-cold no-task-wait baseline time by scale, preferably on a logarithmic time axis. The caption must bind the figure to the frozen build and state 20 instances per scale.

6.2 SRI-3 evaluation

The formal study evaluated deterministic root-only SRI-3, not a learned policy, under the legacy wait-permitted timing rule. Its 1040 strict-cold slots used fresh runtimes and AB/BA alternation. Every slot passed the recorded correctness gate, and scales 5, 10, and 20 passed their promotion gates. Scale 30 did not. The candidate was therefore not promoted, and the production configuration continued to omit this cut family.

Table 2. Formal deterministic root-only SRI-3 promotion decision under the legacy wait-permitted timing rule. A ratio below one favors SRI-3, but promotion follows the complete predefined gate.

Tasks	Correctness	Live/base mean	Paired point estimate	Paired 95% interval	Promotion
5	Pass	1.0035	1.0035	[0.9968, 1.0096]	Pass
10	Pass	0.9811	0.9797	[0.9518, 0.9995]	Pass
20	Pass	0.8052	0.8644	[0.7713, 0.9512]	Pass
30	Pass	1.0877	0.9590	[0.8247, 1.1034]	Fail

The scale-30 mean exceeded the no-cut mean, the paired point estimate did not meet the required threshold, and the interval crossed one. Reporting the favorable median alone would therefore contradict the formal decision. Deterministic cut evidence is included to document the exact engine and its negative promotion result; it is not evidence that learned pricing or branch ranking is effective, and it is not a validation of the revised no-task-wait formulation.

6.3 Legacy exact-state refinements

Two exact state optimizations were audited after the formal SRI-3 decision within the same legacy timing model. Exact nonzero-dual projection retained the full RMP and proof context while binding a projected nonzero context separately. Packed exact-overlap state reduced cut-state storage from 17 to 8 bytes and label-state storage from 168 to 152 bytes, using equality of packed overlap as the dominance condition. These are data-structure facts, not end-to-end memory-scaling claims.

Controlled replay used two recorded pricing states and ten repetitions per state and mode. All paired exact results were equivalent, with zero reduced-cost mismatches. One additional scale-20 end-to-end pair closed to the same objective, 1.893717, in 15.951197 s without cuts and 11.883917 s under the optimized root-only SRI-3 policy. That pair is diagnostic only: it contains one run per mode on one instance, and its source explicitly marks it as non-promotion evidence.

6.4 Learning-guidance evaluation

The learning-result structure is present, but no learning result is currently claimed. All values remain deliberately empty until the corresponding frozen artifacts pass the exactness gates in Section 5.6. The implementation baseline in Table 1 verifies the revised timing semantics and exact closure, but it is not a substitute for the still-pending, strictly paired L0 rows in Table 3.

Table 3. Exact-safety audit for learning variants.

Variant	Exact rows / total	Objective mismatches	RC mismatches	Unsupported proof/pruning decisions	Unresolved deferred-pricing obligations	Candidate-set mismatches
L0	TBD	TBD	TBD	TBD	TBD	TBD
L1	TBD	TBD	TBD	TBD	TBD	TBD
L2	TBD	TBD	TBD	TBD	TBD	TBD

[[TBD-EXP-L0: Insert the frozen no-learning exact control rows and safety summary.]]

[[TBD-EXP-L1: Insert L1 versus L0 paired end-to-end and pricing-work effects, uncertainty, inference overhead, exact-completion effort, and fallback frequency only after Table 3 passes.]]

[[TBD-EXP-L2: Insert L2 versus L1 incremental branch effect, node and candidate-evaluation workload, child pricing workload, inference overhead, and fallback frequency over identical exact-valid candidate sets.]]

[[TBD-EXP-G: Insert held-out/OOD exact closure, calibration, workload, inference overhead, fallback, regressions, and incomplete rows.]]

[[TBD-FIG15: Insert the paired L0/L1/L2 workload and wall-time figure only after M004 rows and uncertainty calculations are frozen. Do not add a directional annotation before activation.]]

[[TBD-FIG16: Insert the inference-overhead, fallback, deferred-pricing, and held-out figure only after M001 defines the split and M005 supplies frozen safety/OOD rows.]]

These placeholders identify the evidence required to test the paper’s central guidance claim. Their position after the explicitly labeled legacy-engine evidence preserves the intended evidence sequence without converting an unfinished experiment into a result. Every activated L0/L1/L2 row must use the verified no-task-wait implementation.

6.5 Seasonal operating phases

[[TBD-EXP-EPOCH: Insert the paired independently frozen mission-epoch instances only after M006 is complete. For every paired task set, report the epoch anchor, four-phase label, mission-window duration, hourly sampling coverage and window-aggregation rule; path-option additions, removals and attribute changes; exact, infeasible, or incomplete terminal status; normalized objective and its three components; reporting-only makespan; energy, shadow exposure and risk; route/trip structure; and any change in the required fleet. Summarize the three anchors of each phase within a paired family before estimating paired phase contrasts and uncertainty intervals. Do not rank phases from unpaired task sets, epoch-specific normalization, or incomplete rows. Report infeasibility separately rather than replacing it with an artificial completion time.]]

This analysis will test sensitivity to the epoch-conditioned environmental summary without altering the mathematical model or exact algorithm. A phase may be described as more favorable only within the tested map, rover, task-family, mission-window, and aggregation settings, and only when the predeclared completion-time evidence supports that statement. The analysis will not establish a universally best lunar exploration season, continuous-time illumination optimality, robustness of one route across epochs, or the performance of an online replanning policy.

6.6 Scalability limits

Bounded strict-cold runs were performed with the legacy wait-permitted implementation on instance 001 at scales 50 and 100 under an effective 8 GiB host-memory limit. Both runs terminated with incomplete pricing searches after reaching the memory limit. Because the search frontier was nonempty, the solver correctly withheld any exact conclusion. Both records passed all checks designed to detect an unsupported proof or pruning decision.

Table 4. Bounded resource-limit evidence under the legacy wait-permitted timing rule.

Tasks	Wall time (s)	Peak RSS (GiB)	Terminal class	Exact conclusion
50	340.135	8.003	Legal incomplete	None
100	300.159	8.001	Legal incomplete	None

These rows demonstrate fail-closed behavior under the recorded limit. They do not prove optimality, infeasibility, or the absence of negative columns at either scale.

7. Discussion and Limitations

7.1 Established evidence

The available evidence establishes both the proof discipline and exact closure of the no-task-wait baseline on the frozen 80-instance scale-5–30 set. The revised implementation preserves one normalized objective across master and pricing, binds the timing policy and instance contents to each proof record, and passes the compact/native timing checks. Historical experiments also record a failed deterministic cut promotion without selective omission and fail-closed termination when resource limits prevented exact completion, but those measurements used the predecessor wait-permitted model. No evidence yet establishes whether learned guidance preserves all revised L0 outcomes or reduces work after overhead and fallback are counted.

[[TBD-DISC-RQ1-RQ5: After EXP-L0/L1/L2/G and EXP-EPOCH are frozen, answer RQ1–RQ5 in order. Each answer must state the measured effect, uncertainty, exact-safety result, fallback condition, and tested scope. Leave each unsupported answer empty until its own activation evidence exists.]]

7.2 Pricing as primary guidance

Pricing is the primary learning target for a structural reason. It is invoked repeatedly while a node is open and explores a resource-rich state space over task order, path alternatives, trips, and active exact context. Earlier discovery of useful columns could alter many later RMP iterations. Branch ranking acts only when a fractional node requires a split, and its effect is mediated by the pricing work induced in the children. This reasoning motivates the L1-before-L2 ablation; it does not predict that L1 or L2 will be faster.

The exact fallback also differs. Pricing can allow finite delay only while every resulting obligation remains explicit and is resolved before closure. Branch ranking can change evaluation order only within an exact-valid candidate set and must use an available exact alternative branch or leave the node incomplete. These interfaces make the two learned actions measurable without giving either action control over proof.

7.3 Operational implications

The model treats prospecting as fleet-level movement and resource allocation: which rover serves each task, how tasks are sequenced across trips, which precomputed path option is chosen, when each trip departs after any depot wait, and when a rover returns and recharges. Prohibiting candidate-site waiting avoids schedules that deliberately arrive early and remain at a prospecting location solely to satisfy a later opening time; temporal flexibility is concentrated at the support depot. If future learning experiments show lower exact-solver effort under unchanged proof conditions, the direct implication will be the ability to evaluate more instances, or the same instances under tighter computational budgets, within the tested setting. It will not imply improved scientific yield, safer physical driving, or a validated operational mission.

[[TBD-DISC-IMPLICATION: Insert a bounded transportation-system implication only after the corresponding L1/L2 effect and failure distribution are frozen. Do not infer mission benefit from deterministic SRI or diagnostic evidence.]]

[[TBD-DISC-PHASE: After M006 and TBD-EXP-EPOCH are frozen, state whether any south-polar seasonal operating phase has a shorter paired normalized science-weighted completion time or reporting-only makespan within the tested conditions. Report disagreement between the two metrics, infeasible or incomplete cases, and uncertainty. Do not generalize the result into a universal best lunar exploration season.]]

7.4 Exactness scope

The guarantee in Theorem 1 depends on an unchanged exact path. Learned orders do not alter task coverage, resource feasibility, objective (10), cut validity, branch construction, or the complete pricing problem. Every delayed negative candidate must be released or covered by exact repricing, and every proof-bearing node bound must bind the active branch/cut context. Removing any of those conditions invalidates the theorem’s conclusion.

For the no-task-wait formulation, the unchanged exact path also includes the departure interval in (3), the temporal equalities in (6a), the pruning rule in (16), and the guarded state comparison in (17). The frozen revised implementation realizes this contract. Theorem 1 may therefore be invoked only for rows that additionally pass the recorded pricing-completion, node-bound, branch-tree, and proof-ledger conditions; all 80 rows in Table 1 did so. An exact status from a predecessor wait-permitted run is not transferable.

Exactness is also conditional on the fixed logical-path solution space. The solver may prove an optimum over all feasible multi-trip routes built from the declared path alternatives, but it does not prove that no better continuous lunar-surface trajectory exists. This limitation persists even if every learning safety gate passes.

The mission-epoch construction does not change this theorem. The same exact solver and proof apply separately after an epoch-conditioned instance has been frozen. The theorem neither identifies a single plan that remains feasible over all epochs nor proves the behavior of a sequence of plans produced after environmental updates.

7.5 Environmental assumptions

The benchmark uses deterministic task, resource, time-window, risk, and path inputs derived from a recorded map pipeline. Terrain and illumination sources support a realistic planning context, while derived risk and resource layers remain model inputs rather than field measurements of future mission outcomes. The approximately **29.5**-Earth-day lunar solar cycle and **346.6**-Earth-day draconic year motivate sampling several declared environmental phases rather than extending one static map indefinitely. Kloos et al. use one-hour samples over **12** lunations and show that seasonally shadowed area depends on local topography and time of year [C063]. The planned protocol therefore places **12** epoch anchors from one southern vernal equinox to the next, approximately **28.9** Earth days apart, and groups three anchors into each south-polar spring, summer, autumn, and winter phase. The routing horizons remain the scale-dependent **16–76** h values. Hourly states over each mission window are summarized before optimization. This separation reduces the temporal aggregation gap but does not make polar illumination invariant or prove that the fixed summary reproduces the exact light state at every departure time. Independent epoch-conditioned instances expose environmental sensitivity without introducing an unsupported within-route light-evolution model. The formulation does not model online map revision, stochastic traction, unexpected hardware faults, or adaptive scientific task creation. It also treats depot standby as supported by base-provided power and thermal control. If that infrastructure cannot be assumed, depot waiting would require explicit standby-energy and thermal-exposure accounting, and the departure interval, recharge coupling, pricing state, and exactness proof would need to be revised.

The three path options per directed edge create a tractable and auditable interface between path preprocessing and exact fleet routing. They also bound transfer. A different path generator, environmental model, or uncertainty representation would define a new solution space and require a new equivalence and proof audit.

7.6 Computational limitations

The no-task-wait exact baseline closes scales 5–30, with the scale-30 distribution still showing substantial computational variation. The only available scale-50 and scale-100 resource-bound runs used the predecessor wait-permitted model. Their legal incompleteness confirms fail-closed behavior for that implementation but does not

quantify the revised model at those scales. New no-task-wait runs are required before drawing a revised-model scalability conclusion.

The principal empirical limitation is more immediate: no frozen training split, selected checkpoint, L0/L1/L2 ablation, or held-out learning package is yet available. Consequently, this working draft makes no claim about learned speed, search effort, robustness, or generalization. Explicit evidence requirements turn that absence into a concrete completion plan rather than an implied result.

7.7 Future work

The no-task-wait implementation and its scale-5–30 verification package are complete. The next empirical steps are to freeze M001–M005 for the correctness-first learning protocol and M006 for the paired seasonal operating-phase analysis. The multi-epoch construction also supplies a conservative route toward operational updating: a rolling-horizon system could refresh the illumination layer, regenerate the path-option records, preserve already executed decisions, and re-solve the remaining tasks with the same exact BPC core. Exactness would continue to apply to each frozen residual instance, but not automatically to the complete adaptive mission policy. A model that changes path attributes inside one route according to its realized departure time would be a different extension and would require new SPPRC transitions, dominance conditions, and a new canonical-route proof. Other extensions include uncertain resource models, heterogeneous rover capabilities, and transfer across map regions.

8. Conclusion

This paper develops a pricing-led, branching-assisted learning-guided exact Branch-Price-and-Cut framework for multi-trip lunar water-ice exploration routing. The formulation selects one-rover multi-trip route columns over a fixed logical-path solution space, prohibits waiting at candidate tasks and en route, and places all schedule adjustment at the support depot through trip-departure intervals. It applies the normalized sum of operating cost, risk, and **0.4** times science-weighted completion time. Learning is restricted to ordering pricing work and ranking exact-valid branch candidates; deterministic cuts, exact pricing completion, exact branch construction, fail-closed incomplete handling, bounds, pruning, and proof records remain on the exact path.

Current evidence establishes exact closure for the frozen 80-instance scale-5–30 no-task-wait baseline. Historical evidence additionally records a formal non-promotion decision for a deterministic root-only SRI-3 candidate and fail-closed resource-limit behavior on the tested scale-50 and scale-100 instances, but those experiments used the predecessor wait-permitted timing rule and are not transferred to the revised model. The L0/L1/L2 learning safety, workload, overhead, and held-out evidence and the paired seasonal-phase result are not yet available.

The framework provides a controlled way to test computational guidance without letting an unfinished learned component redefine the transportation problem or its proof. Its conclusions remain bounded by the declared path alternatives, deterministic environmental inputs, current corpus, and available exact resources. Future experiments must preserve those boundaries while determining whether learned ordering changes exact-solver effort in practice.

Appendix A. Proof Assumptions and Numerical Scope

The proof in Section 4.7 uses the fixed route and master notation of Eqs. (1)–(13) and the exact-component conditions in Eqs. (14)–(23). Lemma 1 requires time-independent path/resource quantities, nonnegative completion-time weights, task-site and en-route waiting to be prohibited, depot waiting to be allowed without a negative cost, and recharge duration to depend only on trip energy. For a fixed task sequence and fixed path options, each task window is translated into an interval for the common trip departure; the intersection with depot availability and the mission horizon must be nonempty. The proof selects its lower endpoint. Independently solving several epoch-conditioned instances preserves these assumptions because hourly environmental states are summarized before optimization and each resulting instance freezes its path quantities. The claim is model-level exactness for that window-aggregated instance, not physical equivalence to every hourly light state. In contrast, a future model with departure-time-dependent illumination, thermal risk, travel, or recharge inside one solve can invalidate this departure-interval construction and therefore requires a new pricing state and proof.

Lemma 3 requires an optimal RMP, one audited reduced-cost definition, extension-preserving pricing reductions, exhaustive proof-bearing pricing under the active branch/cut context, and no unresolved deferred-pricing obligation. Lemma 4 requires only valid root SRI-3 rows and exact same-route/different-route child construction. If a fractional node has no available exact disjunction, it remains incomplete. Theorem 1 additionally requires an exact-feasible incumbent, a closed tree, valid node bounds, verified proof records, and no open or unresolved node.

The mathematical equalities (25) and (27) use exact arithmetic. Executable conclusions are interpreted within their recorded LP, reduced-cost, bound, integrality, and feasibility tolerances. These tolerances, together with the frozen task graph, path alternatives, objective schema, branch/cut context, and audited proof records, form part of the reported proof scope.

The frozen revised executable prohibits early-arrival waiting at candidate tasks. Its proof scope binds the displayed no-wait transitions, depot-departure logic, path preprocessing, label dominance, exact-completion coverage, numerical tolerances, instance content, and engine hash. The predecessor wait-permitted executable and its historical runs remain outside that scope.

Appendix B. Scope of Root-Only SRI-3 Evidence

Root-only SRI-3 is a deterministic exact-engine component governed by the predefined experimental separation and retention policy used in this paper. Descendant nodes inherit admitted root cuts but perform no new SRI separation. Existing correctness evidence does not by itself establish a runtime benefit, and later projection, packing, replay, or benchmark-only evidence cannot be promoted without its own frozen design and acceptance manifest. The deterministic cut-effect and state-refinement experiments reported in Sections 6.2 and 6.3 used the legacy wait-permitted timing rule. The revised baseline in Section 6.1 uses root-only SRI-3 as a fixed control component but is not a paired estimate of that cut family’s effect.

Appendix C. Exploratory Evidence for the Optimized Candidate

The optimized deterministic root-only SRI-3 candidate completed a 160-slot, single-repeat paired benchmark with all scale-local correctness gates passing. Paired geometric-mean SRI-3/base ratios were approximately 0.9912, 0.9860, 0.8565, and 0.8771 at scales 5, 10, 20, and 30. The source marks the run as benchmark-only: the formal design was incomplete, not all scales satisfied the promotion criterion, and a change to the default production policy was not permitted. These values are therefore exploratory, not formal promotion or learning results, and not evidence for the no-task-wait formulation.

Appendix D. Resource-Limited Runs

The scale-50 and scale-100 bounded rows in Table 4 are legal incomplete records under the effective 8 GiB host limit and the legacy wait-permitted timing rule. Their proof blockers are the resource limit, not a claim of infeasibility. No mature pool, external probe, manual column, or resume was used, and the recorded false-exact count is zero.

Appendix E. Pending Learning Artifacts

[[TBD-ABS-RESULT: After the paired L0/L1/L2 evidence and every exact-safety gate are frozen, replace the abstract's pending-validation sentence with one evidence-bounded result sentence covering the learning effect, inference overhead, exact-fallback frequency, and uncertainty.]]

[[TBD-M001: Dataset, split, target, and leakage manifest.]]

[[TBD-M002: Pricing checkpoint, schema, training, and inference package.]]

[[TBD-M003: Branch checkpoint, valid-candidate labels, and inference package.]]

[[TBD-M004: L0/L1/L2 paired run manifests, rows, summaries, and uncertainty.]]

[[TBD-M005: Exact-safety, overhead, fallback, held-out, and OOD package.]]

[[TBD-M006: Paired mission-epoch instance and four-phase comparison package.]]

Editorial Reference-Key Map (Working Draft Only)

The following keys are locked for Phase 4. Final reference formatting and BibTeX export remain a later production task.

- **C001.** You, Yang, Wang, and Yin. “Two-Stage Learning to Branch in Branch-Price-and-Cut Algorithms for Solving Vehicle Routing Problems Exactly.” *Operations Research*, 2026. <https://doi.org/10.1287/opre.2023.0615>
- **C002.** Abouelrous et al. “Reinforcement Learning for Solving the Pricing Problem in Column Generation for Routing.” *Operations Research Perspectives*, 2025. <https://doi.org/10.1016/j.orp.2025.100364>
- **C003.** Wang et al. “Learning to Branch in Combinatorial Optimization With Graph Pointer Networks.” *IEEE/CAA Journal of Automatica Sinica*, 2024. <https://doi.org/10.1109/JAS.2023.124113>
- **C008.** Cappart et al. “Combinatorial Optimization and Reasoning with Graph Neural Networks.” *Journal of Machine Learning Research*, 24(130), 2023. <https://jmlr.org/papers/v24/21-0449.html>
- **C009.** Qu et al. “Enhancing Column Generation by Reinforcement Learning-Based Hyper-Heuristic for Vehicle Routing and Scheduling Problems.” *Computers & Industrial Engineering*, 2025. <https://doi.org/10.1016/j.cie.2025.111138>
- **C020.** Cabrera, Cordeau, and Mendoza. “Solving the Park-and-Loop Routing Problem by Branch-Price-and-Cut.” *Transportation Research Part C*, 2023. <https://doi.org/10.1016/j.trc.2023.104369>
- **C021.** Bezzi, Ceselli, and Righini. “A Route-Based Algorithm for the Electric Vehicle Routing Problem with Multiple Technologies.” *Transportation Research Part C*, 2023. <https://doi.org/10.1016/j.trc.2023.104374>
- **C022.** Sakarya et al. “Two-Echelon Prize-Collecting Vehicle Routing with Time Windows and Vehicle Synchronization: A Branch-and-Price Approach.” *Transportation Research Part C*, 2025. <https://doi.org/10.1016/j.trc.2024.104987>
- **C023.** Xu et al. “An Exact Algorithm for Unpaired Pickup and Delivery Vehicle Routing Problem with Multiple Commodities and Multiple Visits.” *Transportation Research Part C*, 2024. <https://doi.org/10.1016/j.trc.2024.104488>
- **C025.** Qin and Pournaras. “Coordination of Drones at Scale: Decentralized Energy-Aware Swarm Intelligence for Spatio-Temporal Sensing.” *Transportation Research Part C*, 2023. <https://doi.org/10.1016/j.trc.2023.104387>
- **C028.** Lera-Romero, Miranda Bront, and Soullignac. “A Branch-Cut-and-Price Algorithm for the Time-Dependent Electric Vehicle Routing Problem with Time Windows.” *European Journal of Operational Research*, 2024. <https://doi.org/10.1016/j.ejor.2023.06.037>
- **C029.** Nafstad, Desaulniers, and Stålhane. “Branch-Price-and-Cut for the Electric Vehicle Routing Problem with Heterogeneous Recharging Technologies and Nonlinear Recharging Functions.” *Transportation Science*, 2025. <https://doi.org/10.1287/trsc.2024.0725>
- **C030.** Yuan, Cui, and Baldacci. “An Exact Algorithm for a Mobile Production Vehicle Routing Problem.” *Transportation Research Part E*, 2025. <https://doi.org/10.1016/j.tre.2025.104255>
- **C041.** Chen, Jackson, Allard, and Beltrame. “Path Planning Algorithm for a South Pole Lunar Rover Mission.” *Acta Astronautica*, 2025. <https://doi.org/10.1016/j.actaastro.2025.07.059>
- **C042.** Lamarre, Malhotra, and Kelly. “Safe Mission-Level Path Planning for Exploration of Lunar Shadowed Regions by a Solar-Powered Rover.” *IEEE Aerospace Conference*, 2024. <https://doi.org/10.1109/AERO58975.2024.10521136>
- **C044.** Mazarico et al. “Sunlit Pathways between South Pole Sites of Interest for Lunar Exploration.” *Acta Astronautica*, 2023. <https://doi.org/10.1016/j.actaastro.2022.12.023>

- **C054.** Sefton-Nash et al. “Targeting Intermittently Sunlit Areas With Thermal Stability for Buried Water Ice in the South Polar Region of the Moon.” *Journal of Geophysical Research: Planets*, 2026. <https://doi.org/10.1029/2025JE008985>
- **C055.** NASA. “VIPER Lunar Operations.” <https://science.nasa.gov/mission/viper/lunar-operations/>
- **C059.** Desaulniers, Galindo Pecin, and Contardo. “Selective Pricing in Branch-Price-and-Cut Algorithms for Vehicle Routing.” *EURO Journal on Transportation and Logistics*, 2019. <https://doi.org/10.1007/s13676-017-0112-9>
- **C060.** Poggi and Uchoa. “New Exact Algorithms for the Capacitated Vehicle Routing Problem.” In *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed., 2014. <https://doi.org/10.1137/1.9781611973594.ch3>
- **C061.** Zhou et al. “Chang’E-5 Samples Reveal High Water Content in Lunar Minerals.” *Nature Communications*, 13, 5336, 2022. <https://doi.org/10.1038/s41467-022-33095-1>
- **C062.** He et al. “A Solar Wind-Derived Water Reservoir on the Moon Hosted by Impact Glass Beads.” *Nature Geoscience*, 16, 294–300, 2023. <https://doi.org/10.1038/s41561-023-01159-6>
- **C063.** Kloos, Moores, Sangha, Nguyen, and Schorghofer. “The Temporal and Geographic Extent of Seasonal Cold Trapping on the Moon.” *Journal of Geophysical Research: Planets*, 124, 1935–1944, 2019. <https://doi.org/10.1029/2019JE006003>
- **C064.** Wei, Li, Zhang, Tian, Jiang, Wang, and Ma. “Illumination Conditions near the Moon’s South Pole: Implication for a Concept Design of China’s Chang’E-7 Lunar Polar Exploration.” *Acta Astronautica*, 208, 74–81, 2023. <https://doi.org/10.1016/j.actaastro.2023.03.022>